



A novel adversarial deep TSK fuzzy classifier with its inverse-free fast training

Limin Mao¹ · Hangming Shi² · Yongxin Chou¹ · Jinliang Cong¹ · Mingli Lu¹ · Zekang Bian³ · Suhang Gu^{1,2,4} 

Received: 26 July 2024 / Accepted: 18 April 2025
© The Author(s) 2025

Abstract

Deep learning is one of the most popular machine learning methods, and has been used in many applications. However, conventional deep learning is a fully deterministic model that has difficulty reducing data uncertainty while shedding no light on model transparency. To this end, a novel adversarial deep Takagi–Sugeno–Kang (TSK) fuzzy classifier (AD-TSK), which is characterized by its novel stacked structure and inverse-free fast training, is proposed in this paper. After taking the TSK fuzzy classifier as the subclassifier in each layer of AD-TSK, proactive adversarial example learning is designed based on the adversarial attacks on both the features and labels of the inputs for each subclassifier to generate the adversarial TSK fuzzy classifier (ATSK) with increased robustness to noise. Moreover, the generalization capability of AD-TSK can be guaranteed by its novel stacked structure constructed using the dropout-based gradients of each ATSK to update the original inputs along the stacked structure in a layer-by-layer manner. In particular, by sharing the merits of the Schur complement and Sherman–Morrison formula, the consequent parameters of all fuzzy rules in each ATSK can be determined without the matrix inversion operation, and the corresponding optimal number of fuzzy rules can be optimized based on the previous consequent parameters of all fuzzy rules. This can facilitate the inverse-free fast deep learning algorithm of AD-TSK while maintaining interpretability. Extensive experiments on fourteen benchmark datasets from the UCI and KEEL data repositories and the corresponding statistical test results demonstrate the effectiveness of the proposed AD-TSK as well as its inverse-free fast training. The source code of AD-TSK classifier can be downloaded from <https://github.com/gusuhang10/AD-TSK>.

Keywords TSK fuzzy classifier · Interpretability · Deep fuzzy system · Deep learning · Fast learning

Introduction

In recent decades, owing to their powerful data learning and feature representation capabilities [1–5], deep learning based on neural network frameworks has been a popular research topic in machine learning. However, conventional deep neural networks are deterministic models and cannot effectively handle the uncertainty inherent in raw data [2, 4, 5]. In particular, the internal structure of a conventional deep neural network looks like a black box, which makes it difficult for users to understand how the parameter statuses of all hidden

✉ Suhang Gu
gusuhang09@163.com

Limin Mao
maolm@cslg.edu.cn

Hangming Shi
421133185@qq.com

Yongxin Chou
cslgchouyx@cslg.edu.cn

Jinliang Cong
congjinliang@cslg.edu.cn

Mingli Lu
lumli@cslg.edu.cn

Zekang Bian
bianzekang@163.com

¹ School of Electrical Engineering and Automation, Changshu Institute of Technology, 110 Hushan Road, Suzhou 215500, Jiangsu, China

² School of Intelligent Manufacturing, Changzhou University, 21 Gehu Middle Road, Changzhou 213164, Jiangsu, China

³ School of AI and Computer Science, Jiangnan University, No. 18000 Lihu Road, Wuxi, China

⁴ Department of Computing, The Hong Kong Polytechnic University, 11 Yucai Road, Hung Hom, Kowloon, Hong Kong, China

nodes change. Moreover, in intelligent health care, financial management and decision-making, autonomous driving, intelligent traffic management, etc., special requirements are imposed regarding the transparency and interpretability of models for users [6]. In addition, noise is usually present in raw data, and how to handle this noise effectively to guarantee the generalization capability of machine learning models has gradually gained widespread attention [7–9]. Next, we review some related works and then describe the contributions of this paper.

Existing works

To improve the interpretability of deep learning models with robustness to noise data, several deep fuzzy models have been developed and can be generally categorized into two types. The first type integrates fuzzy logic technology and deep neural networks such that the integrated versions can share the advantages of both of methods. For example, Deng et al. [2] incorporated fuzzy logic technology into deep neural networks and proposed a fusion-based deep fuzzy neural network to obtain fuzzy and neural network representations of knowledge and better handle uncertainty in raw data. Salimi-Badr et al. [4] developed a new self-organizing fuzzy neural network for learning and imitating habitual sequential tasks. This method aims to learn and output sequential task information through fuzzy rules to enable neural networks to output reasonable and more easily understandable results. In [5], an iterative fuzzy radial basis function-based neural network was designed. Since the parameters in the radial basis functions can be iteratively calculated via the fuzzy c-means (FCM) algorithm, the designed deep learning model in [5] could adapt to the distributions of samples derived from different classes, resulting in improved classification performance and traceability for the recognition results [5]. In [8], by taking the output errors of the deep neural network model as the inputs of two fuzzy logic systems, the generated two groups of fuzzy outputs were used to adaptively adjust the model parameters to obtain a noise-immune fuzzy neural network. In [10], a fuzzy version of the restricted Boltzmann machine (RBM) was designed by replacing the parameters in the original RBM model with fuzzy numbers to achieve robust classification performance on datasets with noise. In [11], owing to the introduction of both probabilistic and fuzzy logic techniques, the proposed probabilistic fuzzy neural network with recurrent feedback could effectively accommodate noise data to address uncertainties. Although the aforementioned deep neural network models have achieved improved classification performance, especially in terms of the interpretability of their decisions, the application of fuzzy logic technology is limited to the fuzzification of input data [2] or some neural network model parameters [5] rather than the full training and classification

procedures. Therefore, the above deep fuzzy neural network models have limited interpretability, as shown in Table 1.

The second type addresses deep fuzzy systems with stacked structures based on the Takagi–Sugeno–Kang (TSK) fuzzy system [6] to achieve high interpretability of deep learning models while maintaining good generalizability [12–18]. Specifically, with the stacked generalization principle [19], the deep fuzzy systems in [12–18] were designed by first taking the TSK fuzzy classifier in each layer as a sub-classifier. The stacked structures of these deep fuzzy systems [12–18] were then built by cascading the subclassifiers in the following ways: the subclassifier in each layer was trained by randomly selecting input features, and the outputs of the subclassifier in the current layer were simultaneously integrated into those of the subclassifier in the next layer to determine the prediction results [12]; the fuzzy rules in each subclassifier shared interpretable semantic labels for the input features in all layers, and the outputs of the previous subclassifier were simultaneously integrated into the if-parts of the fuzzy rules in the subclassifier of the next layer [13]; the smooth gradient information of the subclassifier outputs was embedded into the current input features via adversarial example learning, and the updated inputs were taken as the inputs of the subclassifier in the next layer [14, 17]; the noise obtained from the random projections of the prediction results produced by the subclassifier in each layer was randomly added to the input features in the current layer, the updated inputs were presented as the inputs of the subclassifier in the next layer [15, 16], and the improved Wang–Mendel fuzzy system with redefined consequent parts of fuzzy rules was taken as the basic unit in each layer, where the inputs were generated by performing shuffling and sliding window operations on the concatenated outputs of the fuzzy system in the previous layer [18]. The stacked structures constructed in the above ways improved the generalization capabilities of the corresponding deep fuzzy systems. Thus, we focus on a stacked structure based on the stacked generalization principle [19] to design the basic framework of the deep fuzzy system proposed in this paper. Moreover, better interpretability can be achieved for these deep fuzzy systems [12–18] by generating interpretable fuzzy rules, where the antecedent fuzzy subsets are randomly selected from fixed fuzzy partitions with interpretable linguistic labels and assigned to each input feature in all layers. However, as shown in Table 1, a matrix inversion operation is required to determine the consequent parameters of the fuzzy rules in each layer, and when a fuzzy rule is added to search for the optimal number of fuzzy rules in each layer, all model parameters need to be recalculated. These two disadvantages cause deep fuzzy systems to have significant computational complexity, in particular, many fuzzy rules are needed for large-scale datasets, or more layers are cascaded in stacked structures to maintain the generalization capability of deep fuzzy systems [12–18]. Although the fast

Table 1 Comparison of the proposed deep fuzzy classifier with existing deep fuzzy classifiers

References	Interpretable model	Adversarial examples learning	Inverse free	Updating parameters repeatedly
Deng et al. [2]	Limited	No	Do not care	Yes
Salimi-Badr et al. [4]	Limited	No	Do not care	Yes
Roh et al. [5]	Limited	No	Do not care	Yes
Dai et al. [8]	Limited	No	Do not care	Yes
Chen et al. [10]	Limited	No	Do not care	Yes
Li et al. [11]	Limited	No	Do not care	Yes
Qin et al. [12]	Yes	No	No	Yes
Zhang et al. [13]	Yes	No	No	Yes
Gu et al. [14]	Yes	Yes	No	Yes
Zhou et al. [15, 16]	Yes	No	No	Yes
Wang et al. [18]	Yes	No	No	Yes
The Proposed	Yes	Yes	Yes	No

“Limited” denotes that the corresponding deep fuzzy model has limited interpretability, and “Do not care” denotes that it does not fall into the considerations of this paper since the focus concerns a deep TSK fuzzy system

version of the deep fuzzy classifier proposed in [17] alleviates the training speed problem by reducing the number of fuzzy rules, it still faces the above two disadvantages of deep fuzzy systems.

In the literature on deep learning, an effective way to handle noise data is the adversarial example learning method [7], which considers some derivatives of the output errors with respect to the inputs as random perturbations to be incorporated into the input features. The updated inputs are subsequently fed to the deep neural networks in the next epoch until satisfactory model performance is attained. Adversarial example learning [7] has been validated from the perspectives of both theoretical analysis and experiments, thus providing a new way of improving the robustness of deep neural networks to adversarial examples, i.e., the noise data contained in a dataset. In essence, updating the inputs in a layer-by-layer manner to construct the stacked structures of the above deep fuzzy systems [12–18] can be interpreted as adding perturbations to the original inputs in a layer-by-layer manner, which falls into the adversarial example learning paradigm [7]. Therefore, we utilize the adversarial example learning method [7] to handle data noise when designing our deep fuzzy system with high interpretability and good generalization capability.

Motivations and contributions of this paper

To date, most deep learning algorithms have been based on neural network frameworks [2, 3, 5]. When a deep learning algorithm is used to train a deep learning model with better generalization capability, it needs to update many connection weights repeatedly and optimally [20], as shown in Table 1. In addition, users cannot comprehend how the states of the connection weights of the hidden nodes change when training

a deep learning model, which means that how a trained deep learning model makes decisions is not interpretable by users. Therefore, on the basis of the stacked generalization principle [19] and the interpretable TSK fuzzy classifier [6], we aim to develop a novel deep fuzzy system with better generalization capability and interpretability in this paper. When designing a deep fuzzy system, we have the following considerations.

1) Noise is often contained in a dataset, and how to handle this noise effectively is a key point that influences the robustness of a deep learning model. Thus, the proposed AD-TSK should have the ability to process noise data inherent in datasets.

2) When the aforementioned deep fuzzy systems are investigated [6, 12–16], the classification performance and interpretability of these deep fuzzy models depend mainly on their stacked structures. Therefore, when the stacked structure of the proposed AD-TSK is constructed on the basis of the stacked generalization principle [19], the core problem is how to cascade the two subclassifiers in the current and next layers by considering how to address the noise contained in the data simultaneously. Consequently, the generalization capability and high interpretability of AD-TSK can be guaranteed.

3) As shown in Eq. 14, a matrix inversion operation is involved in determining the solution for the consequent parameters of all fuzzy rules in a TSK fuzzy system [12, 14, 16]. Thus, when considering a TSK fuzzy system as the basic unit for constructing a stacked structure on the basis of the stacked generalization principle [19], the computational complexity of deep fuzzy systems is significant in terms of the following two aspects. (1) After a fuzzy rule is added to search for the optimal number of fuzzy rules over a certain tuning range, the solution for the consequent parameters for all fuzzy rules must be repeatedly determined, as shown in

Table 1, and (2) the number of layers or the number of fuzzy rules must usually be large to maintain the generalization capability of deep fuzzy models when solving recognition problems on large-scale datasets. Consequently, a fast deep learning algorithm is required for the proposed AD-TSK to achieve rapid training.

By referring to the adversarial example learning method [7] and works on deep fuzzy systems [12–18], a novel proactive adversarial example learning approach is proposed to develop a new stacked structure based on the stacked generalization principle [19], and a novel adversarial deep TSK fuzzy system termed AD-TSK is designed based on the proactive adversarial example learning. The main contributions of this paper are summarized as follows.

1) According to the adversarial example learning in [21, 22], crafting attacks on labels, i.e., label noise or flipping several labels in samples, can improve the performance of machine learning methods. Therefore, we consider the adversarial attacks on both the features and labels of the inputs, and accordingly develop a proactive adversarial example learning method, which differs from the adversarial example learning scheme proposed in [7] based on only the features of the inputs. With the implementation of proactive adversarial example learning, an adversarial TSK fuzzy classifier (ATSK) with increased robustness to noise can be obtained and is considered as the subclassifier in each layer of the proposed AD-TSK to account for the construction of the stacked structure of AD-TSK.

2) On the basis of the stacked generalization principle [19], the designed ATSK serves as a subclassifier in each layer, and by updating the inputs of each ATSK in a layer-by-layer manner using partial gradient information derived from the output errors of each ATSK with respect to its inputs, a novel stacked structure can be achieved to effectively guarantee the generalization capability and high interpretability of the proposed AD-TSK.

3) With the help of the Schur complement and Sherman–Morrison formula [20, 23, 24], the consequent parameters of the fuzzy rules in the ATSK of each layer can be determined without requiring a matrix inversion operation. In particular, when a fuzzy rule is added to search for the optimal number of fuzzy rules, the use of inverse-free operators enables the updates of the consequent parameters of the fuzzy rules in all ATSKs to be completely based on the previous consequent parameters of all fuzzy rules, so the consequent parameters of the fuzzy rules do not need to be recalculated via the inverse operator. Accordingly, the speed of the deep learning algorithm and the fast training process of AD-TSK can be guaranteed.

4) Compared with that of the zero-order TSK fuzzy classifier, the consequent part of each fuzzy rule in the first-order TSK fuzzy classifier is a linear function of the consequents and input features. This means that the output of the first-

order TSK fuzzy classifier is more sensitive to the inputs [12–16]. Thus, unlike the deep fuzzy systems developed on the basis of the zero-order TSK fuzzy classifier [12, 13, 15, 16], when the stacked structure of AD-TSK is constructed, the first-order TSK fuzzy classifier is selected as the basic unit to train ATSK in each layer to ensure a balance between the classification performance and interpretability of AD-TSK.

Outline of this paper

The remaining parts of this paper are organized as follows. In Section II, some preliminaries, including the classic TSK fuzzy system, the Schur complement, and the Sherman–Morrison formula, are briefly described. In Section III, the proposed AD-TSK is explained in detail. In Section IV, the inverse-free fast deep learning algorithm of AD-TSK is given. Section V presents numerical validations of the experimental results by conducting extensive simulations, and Section VI concludes this paper.

Preliminary work

In this section, we provide some background for the construction of the stacked structure and the theoretical derivation of the fast deep learning algorithm of the proposed AD-TSK.

TSK fuzzy classifiers

For the proposed deep fuzzy model, a TSK fuzzy classifier [6] is used as the basic unit to construct the stacked structure. Therefore, we briefly describe the TSK fuzzy classifier as follows. For a TSK fuzzy classifier with R fuzzy rules, the r th fuzzy rule takes the following form:

$$\begin{aligned} Ru^r : & \text{IF } x_1 \text{ is } A_1^r \wedge x_2 \text{ is } A_2^r \wedge \dots \wedge x_d \text{ is } A_d^r, \\ & \text{THEN } f^r(\mathbf{x}) = p_0^r + p_1^r x_1 + \dots + p_d^r x_d, \quad 1 \leq r \leq R \end{aligned} \quad (1)$$

where R denotes the number of fuzzy rules and $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ is an input vector with a dimensionality of d . A_i^r ($1 \leq i \leq d$) denotes the antecedent fuzzy subset, which corresponds to the i th input feature and belongs to the r th fuzzy rule. In addition, the symbol $\wedge$ denotes a fuzzy conjunction operator. After the antecedents of the r th fuzzy rule are determined, the fuzzy set $A^r = [A_1^r, A_2^r, \dots, A_d^r]$ is mapped to the output $f^r(\mathbf{x})$, where the coefficients $p_0^r, p_1^r, \dots, p_d^r$ denote the consequent parameters in the consequent part of the r th fuzzy rule. Here, if only p_0^r is included in the consequent part of each fuzzy rule, i.e., if $f^r(\mathbf{x}) = p_0^r$, the TSK fuzzy system is termed a zero-order TSK fuzzy system. If the consequent part of each fuzzy rule is a linear function of $p_0^r, p_1^r, \dots, p_d^r$, the TSK fuzzy system is termed a

first-order TSK fuzzy system. Furthermore, when a defuzzification strategy, such as weighted average-based aggregation [25], is applied, the overall output of the input $\mathbf{x}$ based on a TSK fuzzy model is expressed as

$$y^o = \sum_{r=1}^R \frac{\mu^r(\mathbf{x})}{\sum_{r'=1}^R \mu^{r'}(\mathbf{x})} f^r(\mathbf{x}) = \sum_{r=1}^R \tilde{\mu}^r(\mathbf{x}) f^r(\mathbf{x}) \quad (2)$$

where $\mu^r(x)$, which is associated with the input $\mathbf{x}$, denotes the firing strength of the r th fuzzy rule, and $\tilde{\mu}^r(\mathbf{x})$ is the normalized firing strength of the r th fuzzy rule. Here, $\mu^r(\mathbf{x})$ and $\tilde{\mu}^r(\mathbf{x})$ are defined as

$$\mu^r(\mathbf{x}) = \prod_{i=1}^d \mu_{A_i^r}(x_i) \quad (3)$$

and

$$\tilde{\mu}^r(\mathbf{x}) = \frac{\mu^r(\mathbf{x})}{\sum_{r=1}^R \mu^r(\mathbf{x})}, \quad (4)$$

respectively. If the commonly used Gaussian membership function is considered, the $\mu_{A_i^r}(x_i)$ of the antecedent fuzzy subset A_i^r in Eq. 3 is expressed as

$$\mu_{A_i^r}(x_i) = \exp\left(-\frac{1}{2} \left(\frac{x_i - c_i^r}{\delta_i^r}\right)^2\right) \quad (5)$$

where the parameters c_i^r and δ_i^r , which are associated with the r th fuzzy rule for the i th input feature x_i , denote the center and width of the Gaussian membership function, respectively. According to [14, 26], if the fuzzy c-means clustering (FCM) algorithm [5] is used to estimate the parameters c_i^r and δ_i^r in Eq. 5, they can be expressed as

$$c_i^r = \frac{\sum_{j=1}^N \mu_{jr} x_{ji}}{\sum_{j=1}^N \mu_{jr}} \quad (6)$$

and

$$\delta_i^r = \frac{h \sum_{j=1}^N \mu_{jr} (x_{ji} - c_i^r)}{\sum_{j=1}^N \mu_{jr}}, \quad (7)$$

respectively. In Eqs. 6 and 7, N denotes the number of inputs, and $1 \leq j \leq N$. μ_{jr} denotes the membership degree of the j th input $\mathbf{x}_j = [x_{j1}, x_{j2}, \dots, x_{jd}]^T$ corresponding to the r th cluster, and it can be determined via FCM [5]. In addition,

the parameter h denotes a scale parameter that can be tuned using several learning strategies, such as cross-validation [26]. For clarity, the Gaussian membership function in Eq. 5 can be substituted with other membership functions, such as the triangular membership function [27], to calculate the antecedent parameters of each fuzzy rule for a TSK fuzzy system.

Once the antecedent parameters of each fuzzy rule are determined for the TSK fuzzy system, we can define the following transformations [26]:

$$\mathbf{x}_e = [1, \mathbf{x}^T]^T, \quad (8)$$

$$\tilde{\mathbf{x}}^r = \tilde{\mu}^r(\mathbf{x}) \mathbf{x}_e, \quad (9)$$

$$\mathbf{u} = \left[(\tilde{\mathbf{x}}^1)^T, (\tilde{\mathbf{x}}^2)^T, \dots, (\tilde{\mathbf{x}}^R)^T \right]^T, \quad (10)$$

$$\mathbf{p}^r = [p_0^r, p_1^r, \dots, p_d^r]^T, \quad (11)$$

$$\mathbf{P}_g = \left[(\mathbf{p}^1)^T, (\mathbf{p}^2)^T, \dots, (\mathbf{p}^R)^T \right]^T. \quad (12)$$

Thus, on the basis of Eqs. 8–12, the output y^o of a TSK fuzzy system for $\mathbf{x}$ is further expressed as a linear model, which is written as

$$y^o = \mathbf{P}_g^T \mathbf{u} \quad (13)$$

where $\mathbf{P}_g$ is the combined consequent vector of all fuzzy rules in the TSK fuzzy system. Considering a dataset $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^T$ with N inputs and the corresponding label set $\mathbf{Y} = [y_1, y_2, \dots, y_N]^T$, several learning strategies, such as minimizing the sum of the squared error [13, 28], implementing ridge regression [14, 29] and performing regularized least-squares regression [30, 31], can be applied to solve problem (13) to determine the solution to $\mathbf{P}_g$. However, when these learning strategies are used to solve problem (13), a matrix inversion operation is required to determine the solution to $\mathbf{P}_g$. For example, when ridge regression [14, 29] is used to learn the consequent parameters of the fuzzy rules in a TSK fuzzy system, $\mathbf{P}_g$ can be expressed as

$$\mathbf{P}_g = \left(\frac{1}{\eta} \mathbf{I} + \mathbf{U}^T \mathbf{U} \right)^{-1} \mathbf{U}^T \mathbf{Y} \quad (14)$$

where η is a regularization constant [12, 14, 16], and $\mathbf{I}$ denotes an identity matrix with proper dimensions. In addition, the antecedent matrix $\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_N]^T$, in which $\mathbf{u}_i$ ($1 \leq i \leq N$) is associated with $\mathbf{x}_i$ in $\mathbf{X}$, can be determined via Eqs. 8–10.

Schur complement and Sherman–Morrison formula

In this subsection, the Schur complement and Sherman–Morrison formula, which are used to develop the fast deep learning algorithm of the proposed AD-TSK model, are briefly described. On the basis of block decomposition, the Schur complement [19, 24] aims to find the inverse of a matrix. Specifically, when matrices **A**, **B**, **C** and **D** have their own inverses, the following equation can be obtained:

$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{C} & \mathbf{D} \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{E} & \mathbf{F} \\ \mathbf{Q} & \mathbf{W} \end{bmatrix} \quad (15)$$

where the four matrices **A**, **B**, **C** and **D** possess proper sizes, and $\mathbf{E} = (\mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C})^{-1}$, $\mathbf{F} = -(\mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C})^{-1}\mathbf{B}\mathbf{D}^{-1}$, $\mathbf{Q} = -\mathbf{D}^{-1}\mathbf{C}(\mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C})^{-1}$ and $\mathbf{W} = \mathbf{D}^{-1}\mathbf{C}(\mathbf{A} - \mathbf{B}\mathbf{D}^{-1}\mathbf{C})^{-1}\mathbf{B}\mathbf{D}^{-1} + \mathbf{D}^{-1}$.

Considering a square matrix **A** that is invertible, and two vectors **s** and **v** possessing proper sizes, the following equation can be obtained via the Sherman–Morrison formula assuming that $1 + \mathbf{v}^T \mathbf{A}^{-1} \mathbf{s} \neq 0$ [23]:

$$(\mathbf{A} + \mathbf{s}\mathbf{v}^T)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{s}\mathbf{v}^T\mathbf{A}^{-1}}{1 + \mathbf{v}^T\mathbf{A}^{-1}\mathbf{s}}. \quad (16)$$

Proposed deep fuzzy model: AD-TSK

In this section, we describe how to construct the proposed deep fuzzy model, its deep learning algorithm and the corresponding computational complexity.

Stacked structure of AD-TSK

This subsection presents the novel stacked structure of the proposed AD-TSK model. For clarity, a first-order TSK fuzzy classifier is used as the basic unit for designing the stacked structure of AD-TSK, as stated in the fourth contribution in the introduction. Before constructing the novel stacked structure of AD-TSK, we should answer the following three questions on the basis of the first two considerations above, and the fourth question corresponding to the third consideration above is left for “Inverse-free fast deep learning algorithm of AD-TSK” section.

- Q1 : In what way should the noisy data in datasets be processed to maintain the robustness of AD-TSK?
- Q2 : In what way are the two subclassifiers in the current and next layers cascaded on the basis of the stacked generalization principle [19]?

Q3 : Do adversarial attacks apply to the testing samples in the prediction phase?

Proactive adversarial example learning (Q1). Adversarial example learning has been proven to be an effective approach for improving the robustness of machine learning models to data noise [7, 32, 33]. In addition, most adversarial example learning approaches usually share characteristics [7, 32, 33] that focus mainly on adversarial attacks on the features of the inputs, i.e., adding imperceptible perturbations to the original input features, when adversarial examples are crafted. However, works using adversarial example learning to secure deep neural networks [21, 22] have demonstrated that adversarial attacks can occur on both the features and labels of the inputs. This means that the adversarial attacks implemented on the features and realistic labels of the inputs should be considered simultaneously. To this end, a proactive adversarial example learning method is proposed in this paper.

Specifically, some training samples are randomly selected from the training dataset, and the realistic labels of these selected training samples are changed to other realistic labels in the label set of the training dataset. In this way, these selected training samples are crafted to be proactive adversarial examples in the training phase, which are then incorporated into the training dataset. In addition, by conducting proactive adversarial example learning, i.e., training a first-order TSK fuzzy classifier [6] on the training dataset including proactive adversarial examples, an ATSK is obtained. Moreover, compared with other types of adversarial attacks on the input features, such as query-based attacks, transfer-based attacks, and optimization-based attacks, gradient-based attacks can be applied to efficiently elaborate the adversarial samples according to [7, 14, 33]. Consequently, the gradients of the output errors induced by ATSK with respect to the training samples, which contain the proactive adversarial examples, are integrated into the features of the training samples. In essence, the proactive adversarial example learning approach considers the adversarial attacks imposed on both the features and labels of the inputs, which makes it different from most adversarial example learning approaches that concern mainly the adversarial attacks on the input features [7, 32, 33].

Dropout-based inputs updated in a layer-by-layer manner (Q2). On the basis of Q1, when the original inputs are updated via gradient-based attacks, all gradient information possessed by the output errors with respect to the inputs is considered, e.g., gradient information is added to the input features in each epoch for deep neural networks [7, 32, 33] and all gradient information is incorporated into the input features in a layer-by-layer manner for a deep TSK fuzzy classifier [14]. Additionally, on the one hand, dropout has been widely used to improve the robustness of deep neural networks by avoiding the coadaptation of feature detectors [34,

35]. On the other hand, equivalence is maintained between fuzzy rule-based systems and extreme learning machines (ELMs), which are feed-forward neural networks [20, 36]. Here, only partial gradient information, which is randomly selected from all gradient information in the output errors induced by ATSK with respect to the inputs, is integrated into the corresponding input features. Consequently, when ATSK is used as the basic subclassifier for constructing the stacked structure of AD-TSK, the inputs of ATSK in the current layer are updated in a dropout manner, and the updated input features serve as the inputs of ATSK in the next layer. In this way, the subclassifiers in the current and next layers are cascaded to form the stacked structure of AD-TSK in a layer-by-layer manner.

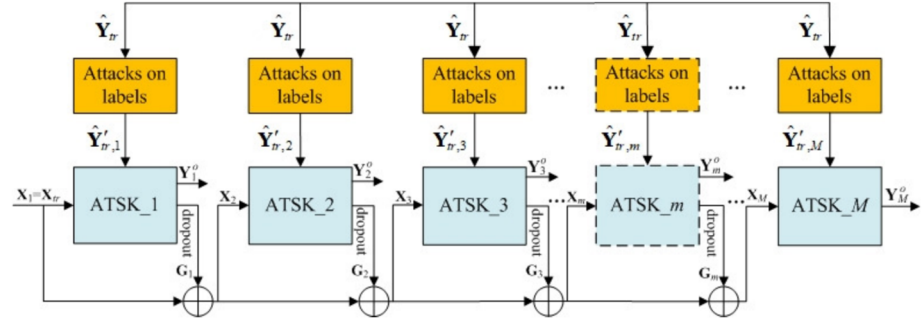
Considering whether adversarial attacks apply to the testing samples in the prediction phase (Q3). In most adversarial example learning approaches, only the training phase involves crafting adversarial examples for adversarial example learning [7, 32, 33]. However, works on utilizing adversarial example learning to secure deep neural networks [21, 22] have indicated that adversarial attacks can also occur on testing samples in addition to training samples. Thus, we consider proactive adversarial example learning in both the training and prediction phases, i.e., proactive adversarial example learning is applied to the testing samples in the prediction phase after a trained AD-TSK model is obtained. For clarity, the labels of the inputs are required in advance to conduct proactive adversarial example learning, and a conventional classifier, such as k-nearest neighbors (KNN) [37] or a support vector machine (SVM) [38], can be employed to label each testing sample. Afterward, proactive adversarial example learning can be conducted on all testing samples along the stacked structure of AD-TSK. Specifically, by implementing adversarial attacks on the labels of the input testing samples, part of the gradient information possessed by the output errors induced by the trained ATSK in the current layer with respect to the testing samples is incorporated into the corresponding features of the testing samples, and the updated testing samples are then input into the trained ATSK in the next layer for prediction. On the basis of proactive adversarial example learning, the original testing samples are updated in a layer-by-layer manner, which is consistent with the process of updating the original training samples along the stacked structure of AD-TSK.

Suppose that we are given a training dataset $\mathbf{X}_{tr} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_{tr}}]^T$, where $\mathbf{x}_i$ ($1 \leq i \leq N_{tr}$) $\in \mathbb{R}^d$, N_{tr} denotes the number of training samples, and d denotes the dimensionality of each training sample. In addition, the realistic label set of $\mathbf{X}_{tr}$ is denoted as $\hat{\mathbf{Y}}_{tr} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{N_{tr}}]^T$. On the basis of the three questions posed above, we construct a novel stacked structure for the proposed AD-TSK, as shown in Fig. 1.

In Fig. 1, the stacked structure of AD-TSK consists of M layers, where ATSK_m ($1 \leq m \leq M$) denotes the subclassifier in the m th layer, and $\mathbf{X}_m = [\mathbf{x}_{m,1}, \mathbf{x}_{m,2}, \dots, \mathbf{x}_{m,N_{tr}}]^T$ and $\mathbf{Y}_m^o = [y_{m,1}^o, y_{m,2}^o, \dots, y_{m,N_{tr}}^o]^T$ denote the input training dataset and the output set of ATSK_m , respectively. In addition, $\hat{\mathbf{Y}}'_{tr,m}$ denotes the label set obtained after adversarial attacks are conducted on $\hat{\mathbf{Y}}_{tr}$, and $\mathbf{G}_m \in \mathbb{R}^{N_{tr} \times d}$ denotes the output gradient matrix of ATSK_m . When the original training dataset $\mathbf{X}_{tr}$ is input into the subclassifier in the first layer, i.e., when $\mathbf{X}_1 = \mathbf{X}_{tr}$, some training samples in $\mathbf{X}_1$ are randomly chosen. Then, adversarial attacks are conducted on the labels of these chosen training samples, i.e., the realistic labels of these chosen training samples are substituted with other labels in $\hat{\mathbf{Y}}_{tr}$, to craft proactive adversarial examples, and the label set of $\mathbf{X}_1$ is changed from $\hat{\mathbf{Y}}_{tr}$ to $\hat{\mathbf{Y}}'_{tr,1} = [\hat{y}'_{1,1}, \hat{y}'_{1,2}, \dots, \hat{y}'_{1,N_{tr}}]^T$. A first-order TSK fuzzy classifier is subsequently trained on $\mathbf{X}_1$, including the proactive adversarial examples, to obtain ATSK_1 . According to the output set $\mathbf{Y}_1^o$ of ATSK_1 , we can calculate the gradients of the output errors induced by $\mathbf{Y}_1^o$ and $\hat{\mathbf{Y}}'_{tr,1}$ with respect to each training sample in $\mathbf{X}_1$. Following that, adversarial attacks are conducted on the features of the training samples in $\mathbf{X}_1$, i.e., partial gradient information is selected from all gradients in a dropout manner for integration into the corresponding features of the training samples in $\mathbf{X}_1$. Consequently, the updated $\mathbf{X}_1$ is regarded as the training dataset $\mathbf{X}_2$, which is denoted as $\mathbf{X}_2 = \mathbf{X}_1 + \lambda \mathbf{G}_1$, of the subclassifier in the second layer, where λ denotes the learning rate. For clarity, “ $\oplus$ ” in Fig. 1 denotes the summation operation, and the elements of $\mathbf{G}_1$ corresponding to the remaining gradient information are set to 0 after adversarial attacks are conducted on the features of the training samples in $\mathbf{X}_1$. Thus, with the proactive adversarial example learning and dropout-based input updating process conducted in a layer-by-layer manner, the stacked structure of AD-TSK can be constructed.

Remark 1 With respect to most deep fuzzy systems [12, 13, 15–18] as shown in Table 1, to improve the robustness of the ATSK in each layer to noise data, we consider adversarial example learning not only for both features and labels of the inputs but also for both the training and prediction phases. In addition, when adversarial example learning is conducted on the input features, partial gradient information is selected from all the gradient information possessed by the output errors induced by ATSK with respect to the inputs in a dropout manner and is integrated into the corresponding input features. In these ways, the generalization capability of AD-TSK can be guaranteed. Moreover, when adversarial example learning is conducted on the labels of the inputs in the training and prediction phases, a small proportion (denoted as β) of the inputs, such as 0.001 or 0.01, is randomly chosen from all inputs to craft adversarial exam-

Fig. 1 Novel stacked structure of AD-TSK



ples, and substituting labels of more inputs with other labels degrades the classification performance of the ATSK in each layer and the same is true of AD-TSK.

After the stacked structure is constructed on the basis of the stacked generalization principle [19], we present the deep learning algorithm of AD-TSK, which consists of training and prediction phases based on proactive adversarial example learning.

Training of AD-TSK

In this subsection, we present how to train the AD-TSK model with the interpretability guarantee according to the stacked structure constructed in the above subsection, and the detailed procedure for training AD-TSK is provided in Algorithm 1.

According to the stacked structure of AD-TSK, a first-order TSK fuzzy classifier is trained on the training dataset $\mathbf{X}_m$, which includes proactive adversarial examples, to obtain ATSK_m in the m th layer. Here, the commonly used Gaussian membership function in Eq. 5 is employed to generate the antecedent parts of all fuzzy rules. In particular, to guarantee the interpretability of ATSK_m , the antecedent part of each fuzzy rule is generated via the strategy of random combining Gaussian membership functions, which has been experimentally proven to be effective in [13–15, 36]. Specifically, the number of fuzzy partitions is first fixed as a small number, such as 3, 5 or 7, and each fuzzy partition corresponds to a Gaussian membership function whose center is fixed with an interpretable linguistic label. Then, the antecedent fuzzy sets of all fuzzy rules are randomly selected from the fixed 5 Gaussian membership functions and are accordingly assigned to the input features. For clarity, the number of fuzzy partitions is set to 5 in this paper according to [12, 36], and the centers of the 5 Gaussian membership functions are fixed at $\{0, 0.25, 0.5, 0.75, 1\}$ with interpretable linguistic labels: $\{\text{very low}, \text{low}, \text{median}, \text{high}, \text{very high}\}$. For example, the linguistic label “very low” is defined as the Gaussian membership function centered at 0, the linguistic label “low” is defined as the Gaussian membership function centered at 0.25, and so on. Moreover, the kernel widths for all Gaussian membership functions can also be randomly

assigned according to [36], and these kernel widths are set to 1 in this paper according to [12]. In this way, the antecedent part of each fuzzy rule is generated by randomly combining the Gaussian membership functions with fixed centers and interpretable linguistic labels, which can guarantee the interpretability of the ATSK in each layer of AD-TSK.

Given a training dataset $\mathbf{X}_{tr}$, as shown in Fig. 1, with the strategy of randomly combining Gaussian membership functions [13–15, 36], the interpretable antecedent parts of all fuzzy rules are obtained for the ATSK in each layer of AD-TSK. Moreover, on the basis of Eqs. 3–10, we can obtain the antecedent matrix $\mathbf{U}_{tr,m}$, which is expressed as

$$\mathbf{U}_{tr,m} = \begin{bmatrix} \tilde{\mu}_m^1(\mathbf{x}_{m,1}) \mathbf{x}_{m,e}^1 & \cdots & \tilde{\mu}_m^{R_m}(\mathbf{x}_{m,1}) \mathbf{x}_{m,e}^{R_m} \\ \vdots & \ddots & \vdots \\ \tilde{\mu}_m^1(\mathbf{x}_{m,N_{tr}}) \mathbf{x}_{m,e}^{N_{tr}} & \cdots & \tilde{\mu}_m^{R_m}(\mathbf{x}_{m,N_{tr}}) \mathbf{x}_{m,e}^{R_m} \end{bmatrix}_{N_{tr} \times (R_m \times (d+1))} \quad (19)$$

for ATSK_m in the m th layer, where $\mathbf{x}_{m,i}$ denotes the i th ($1 \leq i \leq N_{tr}$) training sample in $\mathbf{X}_m$ in the m th layer, and R_m denotes the number of fuzzy rules in ATSK_m . In addition, $\tilde{\mu}_m^r(\mathbf{x}_{m,i})$ ($1 \leq r \leq R_m$) and $\mathbf{x}_{m,e}^i$ are determined via Eqs. 4 and 9, respectively. After the antecedent parts of all fuzzy rules in the ATSK_m in the m th layer are determined, Eq. 14 can be applied to obtain the consequent vector of all fuzzy rules, which is expressed as

$$\mathbf{P}_{g,m} = \left(\frac{1}{\eta} \mathbf{I} + \mathbf{U}_{tr,m}^T \mathbf{U}_{tr,m} \right)^{-1} \mathbf{U}_{tr,m}^T \hat{\mathbf{Y}}_{tr,1}.$$

Additionally, we must determine the output gradient matrix $\mathbf{G}_m$ for ATSK_m in the m th layer. After proactive adversarial example learning is conducted on the labels of the inputs in each layer of AD-TSK, the label set of $\mathbf{X}_m$ in the m th layer is changed from $\hat{\mathbf{Y}}_{tr}$ to $\hat{\mathbf{Y}}'_{tr,m} = [\hat{y}'_{m,1}, \hat{y}'_{m,2}, \dots, \hat{y}'_{m,N_{tr}}]^T$. The output error between $y_{m,i}^o$ and $\hat{y}'_{m,i}$, which corresponds to the i th training sample $\mathbf{x}_{m,i}$ input into ATSK_m in the m th layer, can be expressed as

$$e_{m,i} = (y_{m,i}^o - \hat{y}'_{m,i})^2. \quad (18)$$

Subsequently, the gradient of the output error $e_{m,i}$ with respect to the j th ($1 \leq j \leq d$) feature in $\mathbf{x}_{m,i}$ can be

expressed as

$$\begin{aligned} \frac{\partial e_{m,i}}{\partial x_{m,ij}} = & 2(y_{m,i}^o - \hat{y}_{m,i}') \\ & \sum_{r=1}^{R_m} \left(\tilde{\mu}^r(\mathbf{x}_{m,i}) \sum_{r'=1}^{R_m} \frac{\tilde{\mu}^{r'}(\mathbf{x}_{m,i}) (x_{m,ij} - c_{m,j}^{r'})}{\delta_{m,j}^{r'}} \right. \\ & \left. - \frac{\tilde{\mu}^r(\mathbf{x}_{m,i}) (x_{m,ij} - c_{m,j}^{r'})}{\delta_{m,j}^{r'}} \right) f^r(\mathbf{x}_{m,i}) \\ & + 2(y_{m,i}^o - \hat{y}_{m,i}') \sum_{r=1}^{R_m} \tilde{\mu}^r(\mathbf{x}_{m,i}) p_{m,j}^r \end{aligned} \quad (19)$$

where $p_{m,j}^r$ is the j th element of $\mathbf{p}_m^r = [p_{m,0}^r, p_{m,1}^r, \dots, p_{m,d}^r]^T$, which corresponds to the r th fuzzy rule of ATSK _{m} in the m th layer, and can be determined according to Eq. 11. On the basis of Eq. 19, the (i, j) th element of the gradient matrix $\mathbf{G}_m$ for ATSK _{m} in the m th layer can be expressed as

$$\mathbf{G}_m(i, j) = \frac{\partial e_{m,i}}{\partial x_{m,ij}}. \quad (20)$$

Please note that the gradient information, which is not selected for integration into the input features of ATSK _{m} in the m th layer, is set to 0 for $\mathbf{G}_m$.

Prediction workflow for AD-TSK

On the basis of the third question mentioned in “Training of AD-TSK” section, proactive adversarial example learning is conducted in both the training and prediction phases of AD-TSK. Here, Fig. 2 shows the workflow of the prediction phase of AD-TSK, and the detailed prediction procedure is presented in Algorithm 1. In Fig. 2, $\mathbf{X}_{te} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_{te}}]^T$ denotes the original testing dataset, where N_{te} denotes the number of testing samples, and $\hat{\mathbf{Y}}_{te}$ signifies the label set for $\mathbf{X}_{te}$. In addition, $\bar{\mathbf{X}}_m$ and $\bar{\mathbf{Y}}_m^o$ represent the input testing dataset and output set of the trained ATSK _{m} in the m th layer, respectively, and $\bar{\mathbf{G}}_m$ denotes the output gradient matrix of ATSK _{m} .

Specifically, when the original testing dataset $\mathbf{X}_{te}$ is input into the trained AD-TSK model, the inputs of ATSK₁ in the first layer become $\bar{\mathbf{X}}_1 = \mathbf{X}_{te}$. First, a conventional classifier, such as KNN [37] or an SVM [38], is trained on the training dataset $\mathbf{X}_{tr}$ as shown in Fig. 1, and accordingly, the predictive label set $\hat{\mathbf{Y}}_{te}$ of $\bar{\mathbf{X}}_1$ is output. Then, a small proportion (denoted as β) of the testing samples is randomly selected from $\bar{\mathbf{X}}_1$, and the labels of these selected testing samples are substituted with other labels in $\hat{\mathbf{Y}}_{te}$. Afterward,

the label set of $\bar{\mathbf{X}}_1$ is changed from $\hat{\mathbf{Y}}_{te}$ to $\hat{\mathbf{Y}}_{te,1}'$. According to the R_1 fuzzy rules and $\mathbf{P}_{g,1}$ in the first layer of the training phase, the antecedent matrix $\mathbf{U}_{te,m}$ can be determined for $\bar{\mathbf{X}}_1$ via Eqs. 8–10, and the predictive label set $\bar{\mathbf{Y}}_1^o$ can be determined via ATSK₁ using Eq. 13. Thus, we can obtain the gradients of the output errors between $\bar{\mathbf{Y}}_1^o$ and $\hat{\mathbf{Y}}_{te,1}'$ with respect to $\bar{\mathbf{X}}_1$. Subsequently, partial gradient information is randomly selected from all gradients in a dropout manner to be incorporated into the corresponding features of the testing samples, and the updated testing samples are then regarded as the testing dataset of ATSK₂ in the second layer, i.e., $\bar{\mathbf{X}}_2 = \bar{\mathbf{X}}_1 + \lambda \bar{\mathbf{G}}_1$. For clarity, the elements corresponding to other gradient information in $\bar{\mathbf{G}}_1$, which are not selected from all the gradients to update the testing samples, are set to 0. In this way, the original testing dataset $\mathbf{X}_{te}$ is updated along the stacked structure of AD-TSK in a layer-by-layer manner, and the trained ATSK in each layer outputs its prediction results.

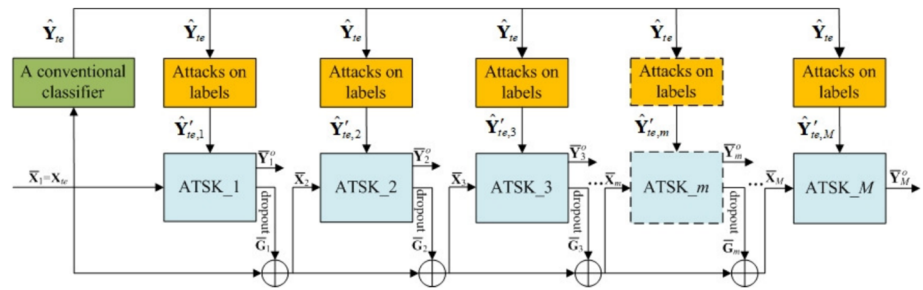
Based on the proactive adversarial example learning, the prediction procedure for AD-TSK remains consistent with the training procedure implemented along the stacked structure of AD-TSK, which guarantees the robustness of ATSK _{m} to the noise in each layer, resulting in the improved generalization capability of AD-TSK.

Deep learning algorithm and the computational complexity of AD-TSK model

On the basis of the stacked structure of AD-TSK, as shown in Fig. 1, and the detailed training and prediction procedures for AD-TSK described above, the deep learning algorithm of AD-TSK is summarized in Algorithm 1, where the computational complexity of the deep learning algorithm is dominated by the training phase of AD-TSK. Thus, we focus on the computational complexity of this phase.

Upon investigating the stacked structure shown in Fig. 1, AD-TSK consists of M ATSKs. Thus, we should first analyze the computational complexity of ATSK in each layer. Moreover, compared with the other steps in the training phase, Steps 4–6 account for a substantial part of the computational complexity incurred when training each ATSK. For ATSK _{m} in the m th layer, $O(5N_{tr}d^2R_m)$ computational complexity is needed to determine the antecedent parts of all fuzzy rules via Eq. 20. Moreover, $O(d^3R_m^3 + N_{tr}d^2R_m^2 + N_{tr}dR_m)$ computational complexity is needed to calculate the consequent vector $\mathbf{P}_{g,m}$ of all fuzzy rules via Eq. 14. Thus, the computational complexity of Step 4 is approximately $O(d^3R_m^3 + N_{tr}d^2R_m^2)$. In addition, the computational complexity of Step 5 is $N_{tr}dR_m$ for outputting the predictive label set $\mathbf{Y}_m^o$ via Eq. 13, and the maximum computational complexity of Step 6 is $O(N_{tr}d^2R_m + N_{tr}dR_m)$ for calculating all gradients of the output errors with respect to the inputs via Eq. 20. Therefore, the computational

Fig. 2 Workflow of the prediction phase of AD-TSK



Algorithm 1 Deep learning algorithm of AD-TSK

Input: $\mathbf{X}_{tr} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_{tr}}]^T$ (training dataset), $\hat{\mathbf{Y}}_{tr} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{N_{tr}}]^T$ (label set of $\mathbf{X}_{tr}$), $\mathbf{X}_{te} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_{te}}]^T$ (testing dataset), M (number of layers), R_m ($1 \leq m \leq M$) (number of fuzzy rules in each layer), η (regularization parameter in Eq. 14), λ (learning rate), ρ (dropout rate), and β (percent of proactive adversarial examples among the inputs).

Output: $\mathbf{P}_{g,m}$ (consequent vector), $\mathbf{Y}_m^o$ (predictive label set for $\mathbf{X}_m$), and $\bar{\mathbf{Y}}_m^o$ (predictive label set for $\bar{\mathbf{X}}_m$).

1: In the training phase:

2: $m \leftarrow 1$ and $\mathbf{X}_m \leftarrow \mathbf{X}_{tr}$.

3: Conduct adversarial attacks on the β of the inputs to craft proactive adversarial examples, and $\hat{\mathbf{Y}}_{tr}$ becomes $\hat{\mathbf{Y}}'_{tr,m}$ in the m th layer.

4: Generate R_m fuzzy rules and train a first-order TSK fuzzy classifier on $\mathbf{X}_m$ with proactive adversarial examples to obtain ATSK_{*m*}. Additionally, compute $\mathbf{U}_m$ and $\mathbf{P}_{g,m}$ using Eqs. 17 and 14, respectively.

5: Determine the predictive label set $\mathbf{Y}_m^o$ using Eq. 13.

6: Calculate the gradients of the errors between $\mathbf{Y}_m^o$ and $\hat{\mathbf{Y}}'_{tr,m}$ with respect to $\mathbf{X}_m$ using Eq. 20, and select gradient information from all gradients in a dropout manner to form $\mathbf{G}_m$.

7: Integrate $\mathbf{G}_m$ into the corresponding input features and regard the updated inputs as the training dataset of the $(m+1)$ th layer, i.e., $\mathbf{X}_{m+1} = \mathbf{X}_m + \lambda \mathbf{G}_m$.

8: If $m > M$, stop the training procedure; otherwise, $m \leftarrow m + 1$, and go to Step 3.

9: In the prediction phase:

10: Based on $\mathbf{X}_{tr}$, determine the label set $\hat{\mathbf{Y}}_{te}$ of $\mathbf{X}_{te}$ using a conventional classifier.

11: $m \leftarrow 1$ and $\bar{\mathbf{X}}_m \leftarrow \mathbf{X}_{te}$.

12: Conduct adversarial attacks on the β of the testing samples to craft proactive adversarial examples, and $\hat{\mathbf{Y}}_{te}$ becomes $\hat{\mathbf{Y}}'_{te,m}$ in the m th layer.

13: Based on the R_m fuzzy rules in Step 4, determine the antecedent matrix $\mathbf{U}_{te,m}$ using Eqs. 8–10.

14: Based on $\mathbf{P}_{g,m}$ in Step 4, determine the predictive label set $\bar{\mathbf{Y}}_m^o$ using Eq. 13.

15: Calculate the gradients of the errors between $\bar{\mathbf{Y}}_m^o$ and $\hat{\mathbf{Y}}'_{te,m}$ with respect to $\bar{\mathbf{X}}_m$, and select gradient information from all gradients in a dropout manner to form $\bar{\mathbf{G}}_m$.

16: Integrate $\bar{\mathbf{G}}_m$ into the corresponding input features and regard the updated inputs as the testing dataset of the $(m+1)$ th layer, i.e., $\bar{\mathbf{X}}_{m+1} = \bar{\mathbf{X}}_m + \lambda \bar{\mathbf{G}}_m$.

17: If $m > M$, stop the testing procedure; otherwise, $m \leftarrow m + 1$, and go to Step 12.

tional complexity of Algorithm 1 of AD-TSK with M layers is approximately $O(M(d^3 R_m^3 + N_{tr} d^2 R_m^2))$.

Inverse-free fast deep learning algorithm of AD-TSK

Q4: Why is a fast deep learning algorithm needed for AD-TSK, and how can it be realized?

Inverse-free fast deep learning algorithm (Q4). The computational complexity of the deep learning algorithm of AD-TSK described above depends linearly on N_{tr} , d^3 , and R_m^3 , where N_{tr} , d and R_m denote the number of training samples, the dimensionality of each training sample, and the number of fuzzy rules in the m th layer, respectively. Thus, when classification problems are solved on datasets with large data scales, high data dimensions, or many necessary fuzzy rules for each ATSK, the computational complexity of AD-TSK becomes significant. Moreover, when searching for the optimal number of fuzzy rules for each ATSK in the training phase of AD-TSK, each ATSK must be retrained after adding a new fuzzy rule, which means that the antecedent and consequent parts of all fuzzy rules need to be recalculated after a new fuzzy rule is added. In particular, calculating each consequent vector $\mathbf{P}_{g,m}$ of all fuzzy rules via Eq. 14 involves a matrix inversion operation, which results in a clear increase in the computational complexity of AD-TSK. In this paper, the Schur complement and Sherman–Morrison formula [20, 23, 24] are introduced to solve the above problem and accordingly achieve an inverse-free fast deep learning algorithm for AD-TSK.

For ATSK_{*m*} with r_m ($1 \leq r_m \leq R_m$) fuzzy rules in the m th layer of AD-TSK, we can use Eqs. 17 and 14 to determine the corresponding antecedent matrix and consequent vector of all fuzzy rules, which are termed $\mathbf{U}_m^{r_m} \in \mathbb{R}^{N_{tr} \times r_m(d+1)}$ and $\mathbf{P}_{g,m}^{r_m} \in \mathbb{R}^{r_m(d+1) \times 1}$, respectively. To conveniently perform the derivation, N_{tr} , r_m , $\mathbf{U}_m^{r_m}$ and $\mathbf{P}_{g,m}^{r_m}$ in ATSK_{*m*} are simplified as N , r , $\mathbf{U}^r$ and $\mathbf{P}^r$, respectively, and the matrix transpose of $\mathbf{U}^r$ is denoted as $\mathbf{H}^r \in \mathbb{R}^{r(d+1) \times N}$, which is written as

$$\mathbf{H}^r = (\mathbf{U}^r)^T. \quad (21)$$

complexity of training ATSK in each layer is approximately $O(d^3 R_m^3 + N_{tr} d^2 R_m^2)$. Accordingly, the computa-

For ATSK_m in the m th layer of AD-TSK, when the number of fuzzy rules increases from r to $r+1$, the antecedent matrix changes from $\mathbf{U}^r$ to $\mathbf{U}^{r+1}$, and $\mathbf{H}^r$ accordingly changes to $\mathbf{H}^{r+1}$. By comparing $\mathbf{U}^{r+1}$ with $\mathbf{U}^r$ via Eq. 17, we observe that $\mathbf{U}^{r+1}$ can be obtained by continuously adding $(d+1)$ columns to $\mathbf{U}^r$ when the number of fuzzy rules changes from r to $r+1$ while retaining the original r fuzzy rules. Similarly, $\mathbf{H}^{r+1}$ can be obtained by continuously adding $(d+1)$ rows to $\mathbf{H}^r$, which is expressed as

$$\mathbf{H}^{r+1} = \begin{bmatrix} \mathbf{H}^r \\ (\mathbf{h}^{r+1})^T \end{bmatrix}_{(r+1)(d+1) \times N} \quad (22)$$

where $\mathbf{h}^{r+1}$ denotes the antecedent matrix of the $(r+1)$ th fuzzy rule, which is expressed as

$$\mathbf{h}^{r+1} = [\mathbf{h}_0^{r+1}, \mathbf{h}_1^{r+1}, \dots, \mathbf{h}_d^{r+1}]_{N \times (d+1)}. \quad (23)$$

Based on Eqs. 22 and 23, when a row is added to $\mathbf{H}^r$, this step is written as

$$\mathbf{H}_{i+1}^r = \begin{bmatrix} \mathbf{H}_i^r \\ (\mathbf{h}_{i+1}^{r+1})^T \end{bmatrix} \quad (24)$$

where $0 \leq i \leq d$, $\mathbf{H}_0^r = \begin{bmatrix} \mathbf{H}^r \\ (\mathbf{h}_0^{r+1})^T \end{bmatrix}$, and $\mathbf{H}^{r+1} = \mathbf{H}_{d+1}^r$.

According to Eqs. 21 and setting $\omega^2 = \frac{1}{\eta}$, the part of Eq. 14 associated with ATSK_m can be expressed as

$$\hat{\mathbf{P}}_{i+1}^r = (\omega^2 \mathbf{I} + \mathbf{H}_{i+1}^r (\mathbf{H}_{i+1}^r)^T)^{-1} \mathbf{H}_{i+1}^r. \quad (25)$$

By introducing an auxiliary matrix

$$\hat{\mathbf{H}}_{i+1}^r = [\mathbf{H}_{i+1}^r \ \omega \mathbf{I}], \quad (26)$$

the pseudo-inverse matrix of $\hat{\mathbf{P}}_{i+1}^r$ in Eq. 25 can be expressed as

$$\begin{aligned} \bar{\mathbf{P}}_{i+1}^r &= (\hat{\mathbf{H}}_{i+1}^r (\hat{\mathbf{H}}_{i+1}^r)^T)^{-1} \hat{\mathbf{H}}_{i+1}^r \\ &= \left[\left(\omega^2 \mathbf{I} + \hat{\mathbf{H}}_{i+1}^r (\hat{\mathbf{H}}_{i+1}^r)^T \right)^{-1} \right. \\ &\quad \left. \hat{\mathbf{H}}_{i+1}^r \omega \left(\omega^2 \mathbf{I} + \hat{\mathbf{H}}_{i+1}^r (\hat{\mathbf{H}}_{i+1}^r)^T \right)^{-1} \right] \end{aligned} \quad (27)$$

Based on Eq. 27, Eq. 25 can be further expressed as

$$\hat{\mathbf{P}}_{i+1}^r = \bar{\mathbf{P}}_{i+1}^r \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i+1) \times N} \end{bmatrix} \quad (28)$$

where $\mathbf{0}$ denotes a matrix whose elements are 0. Similarly, $\hat{\mathbf{P}}_i^r$ is expressed as

$$\hat{\mathbf{P}}_i^r = \bar{\mathbf{P}}_i^r \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i) \times N} \end{bmatrix}. \quad (29)$$

Thus, $\hat{\mathbf{P}}_{i+1}^r$ in Eq. 25 can be determined based on the solution obtained for $\bar{\mathbf{P}}_{i+1}^r$ in Eq. 27, and accordingly, the consequent vector $\mathbf{P}_{g,m}$ of all fuzzy rules can be determined for ATSK_m . In addition, $\hat{\mathbf{H}}_{i+1}^r$ in Eq. 27 can be further expressed as

$$\hat{\mathbf{H}}_{i+1}^r = \begin{bmatrix} \bar{\mathbf{H}}_i^r \\ (\bar{\mathbf{h}}_{i+1}^{r+1})^T \end{bmatrix} \quad (30)$$

where $\bar{\mathbf{H}}_i^r$ and $\bar{\mathbf{h}}_{i+1}^{r+1}$ are written as

$$\bar{\mathbf{H}}_i^r = [\mathbf{H}_i^r \ \omega \mathbf{I}_{(r(d+1)+i) \times 1}] \quad (31)$$

and

$$\bar{\mathbf{h}}_{i+1}^{r+1} = [\mathbf{h}_{i+1}^{r+1} \ \mathbf{0}_{1 \times (r(d+1)+i)} \ \omega], \quad (32)$$

respectively. Moreover, the following equation can be obtained by substituting Eq. 30 into Eq. 27:

$$\bar{\mathbf{P}}_{i+1}^r = \begin{bmatrix} \bar{\mathbf{H}}_i^r (\bar{\mathbf{H}}_i^r)^T & \bar{\mathbf{H}}_i^r \bar{\mathbf{h}}_{i+1}^{r+1} \\ (\bar{\mathbf{h}}_{i+1}^{r+1})^T (\bar{\mathbf{H}}_i^r)^T & (\bar{\mathbf{h}}_{i+1}^{r+1})^T \bar{\mathbf{h}}_{i+1}^{r+1} \end{bmatrix}^{-1} \begin{bmatrix} \bar{\mathbf{H}}_i^r \\ (\bar{\mathbf{h}}_{i+1}^{r+1})^T \end{bmatrix}. \quad (33)$$

Based on the Schur complement [20, 24] in Eq. 15, we can obtain

$$\begin{bmatrix} \bar{\mathbf{H}}_i^r (\bar{\mathbf{H}}_i^r)^T & \bar{\mathbf{H}}_i^r \bar{\mathbf{h}}_{i+1}^{r+1} \\ (\bar{\mathbf{h}}_{i+1}^{r+1})^T (\bar{\mathbf{H}}_i^r)^T & (\bar{\mathbf{h}}_{i+1}^{r+1})^T \bar{\mathbf{h}}_{i+1}^{r+1} \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{E}_0 & \mathbf{F}_0 \\ \mathbf{Q}_0 & \mathbf{W}_0 \end{bmatrix} \quad (34)$$

where $\mathbf{E}_0$, $\mathbf{F}_0$, $\mathbf{Q}_0$, and $\mathbf{W}_0$ are expressed as

$$\mathbf{E}_0 = \left(\bar{\mathbf{H}}_i^r (\bar{\mathbf{H}}_i^r)^T - \frac{\bar{\mathbf{H}}_i^r \bar{\mathbf{h}}_{i+1}^{r+1} (\bar{\mathbf{h}}_{i+1}^{r+1})^T (\bar{\mathbf{H}}_i^r)^T}{(\bar{\mathbf{h}}_{i+1}^{r+1})^T \bar{\mathbf{h}}_{i+1}^{r+1}} \right)^{-1}, \quad (35)$$

$$\mathbf{F}_0 = -\frac{\mathbf{E}_0 \bar{\mathbf{H}}_i^r \bar{\mathbf{h}}_{i+1}^{r+1}}{(\bar{\mathbf{h}}_{i+1}^{r+1})^T \bar{\mathbf{h}}_{i+1}^{r+1}}, \quad (36)$$

$$\mathbf{Q}_0 = \mathbf{F}_0^T, \quad (37)$$

and

$$\mathbf{W}_0 = \frac{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \mathbf{E}_0 \bar{\mathbf{H}}_i \bar{\mathbf{h}}_{i+1}^{r+1}}{\left(\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1}\right)^2} + \frac{1}{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1}}, \quad (38)$$

respectively. By substituting Eq. 34 into Eq. 33, we can obtain the following equation:

$$\begin{aligned} \bar{\mathbf{P}}_{i+1}^r &= \begin{bmatrix} \bar{\mathbf{P}}_{1i+1}^r \\ \bar{\mathbf{P}}_{2i+1}^r \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{E}_0 \bar{\mathbf{H}}_i + \mathbf{F}_0 \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \\ \mathbf{F}_0^T \bar{\mathbf{H}}_i + \mathbf{W}_0 \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \end{bmatrix}. \end{aligned} \quad (39)$$

Therefore, once the two block matrices in Eq. 39, i.e., $\bar{\mathbf{P}}_{1i+1}^r$ and $\bar{\mathbf{P}}_{2i+1}^r$, are determined, $\hat{\mathbf{P}}_{i+1}^r$ in Eq. 28 can be further expressed as

$$\hat{\mathbf{P}}_{i+1}^r = \begin{bmatrix} \hat{\mathbf{P}}_{1i+1}^r \\ \hat{\mathbf{P}}_{2i+1}^r \end{bmatrix}, \quad (40)$$

where

$$\hat{\mathbf{P}}_{1i+1}^r = \bar{\mathbf{P}}_{1i+1}^r \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i+1) \times N} \end{bmatrix}, \quad (41)$$

$$\hat{\mathbf{P}}_{2i+1}^r = \bar{\mathbf{P}}_{2i+1}^r \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i+1) \times N} \end{bmatrix}. \quad (42)$$

In addition, according to the Sherman–Morrison formula [23] in Eq. 16 and supposing that

$\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} - \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \left(\bar{\mathbf{H}}_i \left(\bar{\mathbf{H}}_i\right)^T\right)^{-1} \bar{\mathbf{H}}_i \bar{\mathbf{h}}_{i+1}^{r+1} \neq 0$, $\mathbf{E}_0$ in Eq. 35 can be further expressed as

$$\begin{aligned} \mathbf{E}_0 &= \frac{\left(\bar{\mathbf{H}}_i \left(\bar{\mathbf{H}}_i\right)^T\right)^{-1} \bar{\mathbf{H}}_i \bar{\mathbf{h}}_{i+1}^{r+1} \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \left(\bar{\mathbf{H}}_i \left(\bar{\mathbf{H}}_i\right)^T\right)^{-1}}{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} - \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \left(\bar{\mathbf{H}}_i \left(\bar{\mathbf{H}}_i\right)^T\right)^{-1} \bar{\mathbf{H}}_i \bar{\mathbf{h}}_{i+1}^{r+1}} \\ &\quad + \left(\bar{\mathbf{H}}_i \left(\bar{\mathbf{H}}_i\right)^T\right)^{-1}. \end{aligned} \quad (43)$$

Based on Eq. 43, the two block matrices in Eq. 39, i.e., $\bar{\mathbf{P}}_{1i+1}^r$ and $\bar{\mathbf{P}}_{2i+1}^r$, are further expressed as

$$\begin{aligned} \bar{\mathbf{P}}_{1i+1}^r &= \left(\frac{\check{\mathbf{P}} \bar{\mathbf{h}}_{i+1}^{r+1} \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \check{\mathbf{P}}}{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} - \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \check{\mathbf{P}} \bar{\mathbf{h}}_{i+1}^{r+1}} + \check{\mathbf{P}} \right) \\ &\quad \cdot \frac{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} \mathbf{I} - \bar{\mathbf{h}}_{i+1}^{r+1} \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T}{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1}}, \end{aligned} \quad (44)$$

and

$$\bar{\mathbf{P}}_{2i+1}^r = - \frac{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \bar{\mathbf{P}}_{1i+1}^r}{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1}} + \frac{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T}{\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1}}, \quad (45)$$

respectively, and the auxiliary matrix $\check{\mathbf{P}}$ is expressed as

$$\check{\mathbf{P}} = \begin{bmatrix} \bar{\mathbf{P}}_i^r & \mathbf{0}_{(r(d+1)+i) \times 1} \end{bmatrix}. \quad (46)$$

According to Eqs. 25, 27, 31, 32, and 46, the following equations can be obtained:

$$\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} = \left(\mathbf{h}_{i+1}^{r+1}\right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2, \quad (47)$$

$$\begin{aligned} &\bar{\mathbf{h}}_{i+1}^{r+1} \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \\ &= \begin{bmatrix} \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1}\right)^T & \mathbf{0}_{N \times (r(d+1)+i)} & \omega \mathbf{h}_{i+1}^{r+1} \\ \mathbf{0}_{(r(d+1)+i) \times N} & \mathbf{0}_{(r(d+1)+i) \times (r(d+1)+i)} & \mathbf{0}_{(r(d+1)+i) \times 1} \\ \omega \left(\mathbf{h}_{i+1}^{r+1}\right)^T & \mathbf{0}_{1 \times (r(d+1)+i)} & \omega^2 \end{bmatrix}, \end{aligned} \quad (48)$$

$$\begin{aligned} &\check{\mathbf{P}} \left(\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} \mathbf{I} - \bar{\mathbf{h}}_{i+1}^{r+1} \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \right) \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i+1) \times N} \end{bmatrix} \\ &= \hat{\mathbf{P}}_i^r \left(\left(\left(\mathbf{h}_{i+1}^{r+1}\right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 \right) \mathbf{I} - \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1}\right)^T \right), \end{aligned} \quad (49)$$

$$\begin{aligned} &\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \check{\mathbf{P}} = \left[\left(\mathbf{h}_{i+1}^{r+1}\right)^T \left(\mathbf{H}_i^r\right)^T \hat{\mathbf{P}}_i^r \right. \\ &\quad \left. \omega \left(\mathbf{h}_{i+1}^{r+1}\right)^T \left(\mathbf{H}_i^r\right)^T \left(\omega^2 \mathbf{I} + \mathbf{H}_i^r \left(\mathbf{H}_i^r\right)^T \right)^{-1} \quad \mathbf{0} \right], \end{aligned} \quad (50)$$

$$\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \left(\bar{\mathbf{H}}_i\right)^T \check{\mathbf{P}} \bar{\mathbf{h}}_{i+1}^{r+1} = \left(\mathbf{h}_{i+1}^{r+1}\right)^T \left(\mathbf{H}_i^r\right)^T \hat{\mathbf{P}}_i^r \left(\mathbf{h}_{i+1}^{r+1}\right)^T, \quad (51)$$

$$\begin{aligned} &\left(\left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \bar{\mathbf{h}}_{i+1}^{r+1} \mathbf{I} - \bar{\mathbf{h}}_{i+1}^{r+1} \left(\bar{\mathbf{h}}_{i+1}^{r+1}\right)^T \right) \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i+1) \times N} \end{bmatrix} \\ &= \left(\mathbf{h}_{i+1}^{r+1}\right)^T \left(\mathbf{H}_i^r\right)^T \hat{\mathbf{P}}_i^r \\ &\quad \left(\left(\left(\mathbf{h}_{i+1}^{r+1}\right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 \right) \mathbf{I} - \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1}\right)^T \right), \end{aligned} \quad (52)$$

$$\begin{aligned}
& \tilde{\mathbf{P}}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \left(\mathbf{H}_i^r \right)^T \tilde{\mathbf{P}} \\
& \left(\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} \mathbf{I} - \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \right) \\
& \begin{bmatrix} \mathbf{I}_{N \times N} \\ \mathbf{0}_{(r(d+1)+i+1) \times N} \end{bmatrix} \\
& = \hat{\mathbf{P}}_i^r \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \left(\mathbf{H}_i^r \right)^T \hat{\mathbf{P}}_i^r \\
& \left(\left(\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 \right) \mathbf{I} - \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \right).
\end{aligned} \tag{53}$$

Therefore, the two block matrices in Eq. 40, i.e., $\hat{\mathbf{P}}_{1i+1}^r$ and $\hat{\mathbf{P}}_{2i+1}^r$, can be determined as follows:

$$\begin{aligned}
\hat{\mathbf{P}}_{1i+1}^r &= \frac{\hat{\mathbf{P}}_i^r \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \left(\mathbf{H}_i^r \right)^T \hat{\mathbf{P}}_i^r \left(\left(\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 \right) \mathbf{I} - \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \right)}{\left(\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 - \left(\mathbf{h}_{i+1}^{r+1} \right)^T \left(\mathbf{H}_i^r \right)^T \hat{\mathbf{P}}_i^r \mathbf{h}_{i+1}^{r+1} \right) \left(\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 \right)} \\
&+ \frac{\hat{\mathbf{P}}_i^r \left(\left(\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2 \right) \mathbf{I} - \mathbf{h}_{i+1}^{r+1} \left(\mathbf{h}_{i+1}^{r+1} \right)^T \right)}{\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2},
\end{aligned} \tag{54}$$

and

$$\hat{\mathbf{P}}_{2i+1}^r = - \frac{\left(\mathbf{h}_{i+1}^{r+1} \right)^T \left(\mathbf{H}_i^r \right)^T \hat{\mathbf{P}}_{1i+1}^r}{\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2} + \frac{\left(\mathbf{h}_{i+1}^{r+1} \right)^T}{\left(\mathbf{h}_{i+1}^{r+1} \right)^T \mathbf{h}_{i+1}^{r+1} + \omega^2}. \tag{55}$$

Then, $\hat{\mathbf{P}}_{i+1}^r$ in Eq. 40 can be accordingly determined on the basis of Eqs. 54 and 55, which means that $\hat{\mathbf{P}}_{i+1}^r$ can be obtained based on $\hat{\mathbf{P}}_i^r$.

Remark 2 When the number of fuzzy rules increases from r_m to $(r_m + 1)$ in ATSK_{*m*} during the training of the AD-TSK model via Algorithm 1, the consequent vector $\mathbf{p}_{g,m}^{r_m+1}$ of all fuzzy rules can be determined by continuously calculating Eqs. 54 and 55 $(d + 1)$ times on the basis of $\mathbf{P}_{g,m}^{r_m}$, without a matrix inversion operation being required. For clarity, d denotes the dimensionality of the inputs of ATSK_{*m*} in each layer. Thus, with the help of the Schur complement and the Sherman–Morrison formula [20, 23, 24], we can obtain an inverse-free fast deep learning algorithm to realize fast AD-TSK model training and quickly determine the optimal number of fuzzy rules for ATSK_{*m*} in each layer, which significantly differs from the proposed AD-TSK with the other deep fuzzy systems with stacked structures [12–18] as shown in Table 1.

The computational complexity of the inverse-free fast deep learning algorithm of AD-TSK is dominated by the implementation of Eqs. 54 and 55. Specifically, the computational complexity of implementing Eq. 54 once is approximately $O(N_{tr}(R_m(d + 1) + 1))$, and the computational complexity of implementing Eq. 55 once is also approximately $O(N_{tr}(R_m(d + 1) + 1))$. For clarity, N_{tr} denotes the number of training samples input into ATSK_{*m*}, d represents the dimensionality of the inputs, and R_m denotes the number of fuzzy rules in ATSK_{*m*}. This finding indicates that the computational complexity of the inverse-free fast deep learning algorithm of AD-TSK depends linearly on N_{tr} , d and R_m . However, the computational complexity of Algorithm 1 of AD-TSK, which involves a matrix

inversion operation to determine the consequent vector of all fuzzy rules in ATSK_{*m*}, depends linearly on N_{tr} , d^3 and R_m^3 , as mentioned in “Deep learning algorithm and the computational complexity of AD-TSK model” section. Therefore, compared with Algorithm 1, the inverse-free fast deep learning algorithm enables AD-TSK to solve classification problems on datasets whose data dimensions are high or where many fuzzy rules are needed in each layer.

Experimental results and analysis

In this section, the performance of the proposed AD-TSK model is investigated to confirm its effectiveness, especially its proactive adversarial example learning method, stacked structure based on the stacked generalization principle [19], and inverse-free fast deep learning algorithm.

Comparison algorithms and setups

As shown in Fig. 1, the proposed AD-TSK model is developed by constructing a novel stacked structure on the basis of the stacked generalization principle [19], where the basic unit is derived from a conventional TSK fuzzy classifier [6] trained via adversarial example learning. Thus, we compare the classification ability of AD-TSK with that of the following mainstream algorithms. The standard zero-order

and first-order TSK fuzzy classifiers [39–42], which are two conventional TSK fuzzy classifiers, and are termed 0-TSK-FC and 1-TSK-FC, respectively, for simplicity. The deep fuzzy classifier termed DSA-FC [14], which builds a stacked structure by stacking adversarial interpretable TSK fuzzy classifiers on the basis of the stacked generalization principle [19]. To demonstrate the potential distinctions between AD-TSK and the competing approaches, we also consider an SVM [38] and a continuous dropout-based deep neural network (CD-DNN) [34]. For clarity, when running the SVM in all the experiments, the linear and Gaussian kernels are considered, and the corresponding versions of the SVM are termed the LSVM and GSVM, respectively, for simplicity. In addition, we compare the AD-TSK model trained via the inverse-free fast deep learning algorithm (termed F-AD-TSK) to the AD-TSK model trained via Algorithm 1 to verify whether the former strategy can quickly train the AD-TSK model. The detailed parameter setups of all the compared algorithms are described below.

1) 0-TSK-FC: Gaussian membership functions are chosen to calculate the antecedent parts of all fuzzy rules, and the associated centers and kernel widths are set according to [12]. In addition, the regularization parameter η is tuned over $\{10^{-5}, 10^{-4}, \dots, 10^4, 10^5\}$, and the number of fuzzy rules is searched within the range of [5, 200] with an interval of 5.

2) 1-TSK-FC: All involved parameters are set to be the same as those in 0-TSK-FC.

3) DSA-FC: Gaussian membership functions are considered for calculating the antecedent parts of all fuzzy rules, and their centers and widths as well as the regularization parameter η are set to be same as those in 0-TSK-FC. Owing to the stacked structure, the number of fuzzy rules is tuned over $\{2, 3, \dots, 19, 20\}$ for the subclassifier in each layer. In addition, following [14], the number of layers is set to 5, and the chosen adversarial attack percentage for the training samples of the subclassifier in each layer is tuned over $\{0.001, 0.01, 0.1\}$.

4) LSVM: The penalty parameter c is tuned over $\{10^{-5}, 10^{-4}, \dots, 10^4, 10^5\}$.

5) GSVM: The penalty parameter c is set to be the same as that in the LSVM. In addition, the Gaussian kernel function is used, and the kernel width is tuned over $\{10^{-5}, 10^{-4}, \dots, 10^4, 10^5\}$.

6) CD-DNN: For a fair comparison, the number of hidden layers is comparably set to the number of layers possessed by the proposed AD-TSK model. Similarly, the number of hidden nodes in each hidden layer is also comparably set to the number of fuzzy rules used in the ATSK in each layer of AD-TSK.

7) AD-TSK: Gaussian membership functions are used to calculate the antecedent parts of all fuzzy rules, and their centers and widths as well as the regularization parameter η are set to be the same as those in 0-TSK-FC. Owing to

the stacked structure, the number of fuzzy rules is tuned over $\{2, 3, \dots, 19, 20\}$ for the ATSK in each layer. In addition, the number of layers is set to 5, the learning rate λ is tuned over $\{0.001, 0.01, 0.1\}$, the dropout rate ρ is tuned over $\{0, 0.2, 0.4, 0.6, 0.8\}$, and the percentage of proactive adversarial examples among the inputs for the ATSK in each layer is tuned over $\{0.001, 0.01, 0.1\}$.

8) F-AD-TSK: All involved parameters are set to be the same as those in AD-TSK.

Additionally, 80 percent of the samples in each dataset are randomly selected as training samples, and the remaining samples are used as testing samples. Moreover, for the classification problem, the commonly used performance criterion, i.e., accuracy (ACC) defined as $ACC = \frac{TP+TN}{TP+FP+TN+FN}$ where TP , FP , TN and FN denote the true positive, false positive, true negative and false negative, respectively [20], is adopted to evaluate the classification performance of each comparison algorithm. In all the experiments, the parameters of all comparison algorithms are fairly optimized using the grid search strategy for optimal classification performance. Besides, each comparison algorithm is implemented 10 times, and its average classification performance is reported. All the experiments are performed with MATLAB on a computer possessing an Intel(R) Core(TM) i7-12700F operating at 2.10GHz and 16GB of RAM.

Experiments conducted on benchmark datasets

In this subsection, to examine the feasibility of AD-TSK, fourteen benchmark datasets downloaded from the UCI [43] and KEEL [44] data repositories are used, and their details are summarized in Table 2.

Table 3 presents the detailed classification results produced by all the compared algorithms on all the benchmark datasets, where “—” denotes values that are less than 10^{-4} , and the best classification results obtained on each benchmark dataset are shown in bold. In addition, Table 4 shows the average number of fuzzy rules created when 0-TSK-FC, 1-TSK-FC, DSA-FC, AD-TSK and F-AD-TSK achieve their best classification performances on each benchmark dataset. On the basis of Tables 3 and 4, the following observations can be made.

1) Compared with the other algorithms, the proposed AD-TSK model achieves comparable and even superior classification performance on most benchmark datasets. In particular, AD-TSK outperforms both 0-TSK-FC and 1-TSK-FC while using fewer fuzzy rules for most benchmark datasets. These two observations demonstrate the feasibility of AD-TSK, especially its proactive adversarial example learning-based stacked structure, which can effectively guarantee the generalization capability of AD-TSK.

2) Although AD-TSK and DSA-FC have stacked structures constructed on the basis of the stacked generalization

Table 2 Descriptions of the fourteen utilized benchmark datasets

Datasets	#Samples	#Dimensions	#Classes
Liver(LIV)	345	6	2
Pima(PIM)	768	8	2
RedWineQuality(RED)	1599	11	6
Titanic(TIT)	2201	3	2
Chess(CHE)	3196	36	2
Phoneme(PHO)	5404	5	2
Pageblocks(PAG)	5473	10	5
Twonorm(TWO)	7400	20	2
Nursery(NUR)	12,960	8	5
Magic(MAG)	19,020	10	2
CreditCardClients(CCC)	30,000	23	2
Adult(ADU)	48,841	14	2
Shuttle(SHU)	58,000	9	8
InternetFirewall(INT)	65,532	11	4

principle [19], in most cases, AD-TSK performs better than does DSA-FC, where all gradient information possessed by the output errors induced by the subclassifier in each layer with respect to the inputs is integrated into the input features to update the inputs of each subclassifier in a layer-by-layer manner [14]. This observation validates the effectiveness of selecting partial gradient information from the output errors induced by ATSK with respect to the inputs in a dropout manner to update the inputs of each ATSK in a layer-by-layer manner for AD-TSK.

3) When DSA-FC, AD-TSK and F-AD-TSK achieve their best classification performance, the corresponding stacked structures generally consist of 2 to 5 layers, which is consistent with the conclusion obtained in [14]. This observation also indicates that cascading 5 layers via proactive adversarial example learning to construct the stacked structure of AD-TSK can help AD-TSK perform well in most cases.

4) Compared with AD-TSK, F-AD-TSK can achieve comparable classification performance, which shows that the inverse-free fast deep learning algorithm developed on the basis of the Schur complement and Sherman–Morrison formula [20, 23, 24] does not affect the performance of AD-TSK.

The classification results obtained on benchmark datasets demonstrate the feasibility of AD-TSK, and according to these classification results, we next investigate the performance of AD-TSK through a statistical comparison. To this end, a robust and nonparametric test method [45] is employed, which is briefly described as follows, and its implementation details can be found in [45].

First, before the statistical test method is conducted, we assume that all the compared algorithms have identical performances on all benchmark datasets. However, whether this assumption holds must be confirmed via the Friedman test

[45], for which the statistical value F_F should be calculated. The initial assumption can subsequently be rejected if the value satisfies $F_F > ((N_c - 1), (N_c - 1)(N_d - 1))$, where N_c and N_d denote the numbers of compared algorithms and benchmark datasets, respectively. After the initial assumption is rejected, we further implement the Bonferroni-Dunn test [45], where the critical value CD needs to be determined via $CD = q_\alpha \sqrt{N_c(N_c + 1)/6N_d}$. For clarity, α denotes the significance level, and it is usually set to 0.05 by referring to [45], and accordingly, the specific value of q_α can be found in the table of critical values for post-hoc tests after the Friedman test is conducted [45]. Moreover, we need to rank all the compared algorithms according to their classification performances achieved on all the benchmark datasets (shown in Table 3). In particular, if the difference between the average ranks of two compared algorithms is greater than CD , there is a significant difference between the classification performances of these two compared algorithms [45].

According to Table 3, the numbers of compared algorithms and benchmark datasets are $N_c = 8$ and $N_d = 14$, respectively. In addition, Table 5 presents the ranks of each compared algorithm on all benchmark datasets in terms of their classification performances (shown in Table 3). On the basis of the average ranks achieved by each compared algorithm on all benchmark datasets, the statistical value F_F of the Friedman test [45] is $F_F \approx 12.53$. Moreover, according to the F-distribution [45], $F(7, 91) \approx 2.11$. Therefore, the above assumption that all the compared algorithms perform equally on all the benchmark datasets is rejected because $F_F > F(7, 91)$. Additionally, according to the table of critical values for post-hoc tests [45], $q_{0.05} = 2.690$ with $\alpha = 0.05$ and $N_c = 8$. Thus, the critical value CD becomes $CD \approx 2.49$. Consequently, with respect to the other compared algorithms, AD-TSK and F-AD-TSK rank first and

Table 3 Classification accuracies achieved by all compared algorithms on all benchmark datasets (mean and standard deviation values %)

Datasets	0-TSK-FC	1-TSK-FC	DSA-FC	LSVM	GSVM	CD-DNN	AD-TSK	F-AD-TSK
LIV	72.17 ± 0.66	75.07 ± 1.14	78.33 ± 0.13	72.46 ± 4.37	71.88 ± 3.99	75.17 ± –	81.88 ± 1.57	80.00 ± 2.44
PIM	71.69 ± 0.55	73.12 ± 0.69	80.26 ± 0.60	63.51 ± 3.53	65.06 ± 3.73	82.08 ± 1.96	77.53 ± 0.82	78.12 ± 1.59
RED	61.28 ± 0.77	62.18 ± 0.29	63.49 ± 0.53	60.38 ± 1.11	60.06 ± 2.20	60.88 ± 0.88	65.00 ± 0.74	63.09 ± 1.07
TIT	78.79 ± 0.35	78.91 ± –	79.95 ± –	77.42 ± 1.18	78.96 ± 1.84	79.84 ± 0.16	82.54 ± 0.28	80.05 ± 0.25
CHE	95.92 ± 0.36	99.00 ± 0.13	97.67 ± 0.74	96.92 ± 0.63	97.48 ± 0.85	97.61 ± 0.69	98.39 ± 0.23	98.02 ± 0.47
PHO	83.06 ± 0.27	85.24 ± 0.15	82.18 ± 0.36	77.50 ± 1.15	77.12 ± 1.02	80.82 ± 0.08	87.42 ± 0.28	85.85 ± 0.85
PAG	90.45 ± 0.50	93.23 ± 0.17	93.36 ± 0.18	96.69 ± 0.41	97.20 ± 0.49	95.15 ± 0.23	96.57 ± 0.10	96.53 ± 0.16
TWO	96.96 ± 0.23	97.83 ± 0.07	98.04 ± 0.11	98.11 ± 0.31	97.94 ± 0.34	97.92 ± 0.04	98.20 ± 0.07	98.14 ± 0.10
NUR	39.05 ± 0.32	42.44 ± 0.02	49.78 ± 1.86	81.01 ± 0.56	80.86 ± 0.75	94.49 ± 0.71	84.81 ± 1.74	77.42 ± 2.45
MAG	86.00 ± –	86.34 ± 0.19	88.97 ± 0.11	86.38 ± 1.69	86.46 ± 0.83	88.88 ± 1.56	90.48 ± 0.21	89.88 ± 0.23
CCC	77.66 ± 0.02	78.53 ± 0.10	78.61 ± 0.01	77.57 ± 0.65	78.18 ± 0.35	80.92 ± 1.12	78.59 ± 0.09	78.23 ± 0.16
ADU	78.98 ± 0.03	80.18 ± 0.08	80.26 ± 0.28	79.55 ± 0.27	79.29 ± 0.48	82.42 ± –	80.92 ± 0.07	80.23 ± 0.15
SHU	78.39 ± –	78.46 ± –	79.02 ± –	96.78 ± 0.14	96.86 ± 0.22	99.01 ± –	99.20 ± –	98.80 ± 0.03
INT	96.17 ± 0.03	97.21 ± 0.02	92.14 ± 0.13	99.08 ± 0.08	98.99 ± 0.15	97.62 ± 0.01	99.56 ± 0.03	99.48 ± 0.10

Table 4 Average number of fuzzy rules produced when 0-TSK-FC, 1-TSK-FC, DSA-FC, AD-TSK and F-AD-TSK achieve their best classification performances on each benchmark dataset

Datasets	0-TSK-FC	1-TSK-FC	DSA-FC	AD-TSK	F-AD-TSK
LIV	23.0	16.5	40.5(4)	11.2(2)	11.3(2)
PIM	78.0	21.0	28.8(3)	10.6(2)	10.3(2)
RED	92.0	11.5	20.5(3)	12.2(2)	14.0(2)
TIT	20.0	5.0	8.6(2)	10.3(3)	8.2(3)
CHE	164.0	121.5	28.0(2)	29.4(2)	37.2(2)
PHO	125.5	63.0	45.6(3)	15.6(4)	17.0(5)
PAG	144.5	27.0	18.2(3)	12.9(5)	14.5(5)
TWO	179.5	7.0	15.3(5)	5.8(2)	11.5(2)
NUR	161.5	66.0	43.4(3)	13.9(5)	18.9(5)
MAG	65.0	54.0	34.2(3)	10.5(5)	12.1(4)
CCC	126.5	57.5	24.5(5)	48.7(4)	43.3(5)
ADU	72.5	8.0	18.5(2)	12.9(2)	16.3(4)
SHU	48.5	16.5	10.0(5)	17.0(3)	19.8(5)
INT	147.0	59.5	11.5(5)	15.7(4)	15.1(5)

Values in parentheses denote the numbers of layers utilized when DSA-FC, AD-TSK and F-AD-TSK achieve their best classification performances on each benchmark dataset

second respectively on all benchmark datasets, which is consistent with the classification results presented in Table 3. Moreover, the performance of AD-TSK significantly differs from that of 0-TSK-FC, 1-TSK-FC, the LSVM and the GSVM because the differences between the average rank of AD-TSK and those of these four compared algorithms are 5.36, 3.58, 3.93, and 3.86, respectively, which are greater than the critical value CD . The same result can be obtained according to the differences between the average ranks of F-AD-TSK and 0-TSK-FC, the LSVM and the GSVM. These results demonstrate the potential superiority of AD-TSK as a deep fuzzy classifier for solving classification tasks. In addition, the difference between the average ranks of AD-TSK and DSA-FC, which is one of the deep fuzzy systems [12–

18], is less than the critical value CD which may be attributed to that these two deep fuzzy classifiers have stacked structures where the TSK fuzzy classifier [6] is chosen as the basic unit in each layer. However, compared with DSA-FC, AD-TSK achieves better average rank on all benchmark datasets. This proves that AD-TSK has better generalization capability which effectively verifies the feasibility of selecting partial gradient information from the output errors induced by the ATSK with respect to the inputs in a dropout manner to update the inputs of each ATSK in a layer-by-layer manner and accordingly developing the stacked structure for AD-TSK. Moreover, F-AD-TSK can also achieve better average rank than DSA-FC on all benchmark datasets. This demonstrates the effectiveness of both the proactive adversarial example

Table 5 Ranks of the comparison algorithms on all benchmark datasets in terms of their classification performances (shown in Table 3)

Datasets	0-TSK-FC	1-TSK-FC	DSA-FC	LSVM	GSVM	CD-DNN	AD-TSK	F-AD-TSK
LIV	7	5	3	6	8	4	1	2
PIM	6	5	2	8	7	1	4	3
RED	5	4	2	7	8	6	1	3
TIT	7	6	3	8	5	4	1	2
CHE	8	1	4	7	6	5	2	3
PHO	4	3	5	7	8	6	1	2
PAG	8	7	6	2	1	5	3	4
TWO	8	7	4	3	5	6	1	2
NUR	8	7	6	3	4	1	2	5
MAG	8	7	3	6	5	4	1	2
CCC	7	4	2	8	6	1	3	5
ADU	8	5	3	6	7	1	2	4
SHU	8	7	6	5	4	2	1	3
INT	7	6	8	3	4	5	1	2
<i>AveRank</i>	7.07	5.29	4.07	5.64	5.57	3.64	1.71	3.00

learning-based stacked structure and the inverse-free fast deep learning algorithm. For AD-TSK and F-AD-TSK, the difference between their average ranks is less than the critical value CD which indicates that the inverse-free fast deep learning algorithm does not affect the generalization capability of F-AD-TSK from the perspective of a statistical test.

Parameter analysis

To further investigate the proposed AD-TSK model, we also observe how the parameters β , λ and ρ , which are the percentage of proactive adversarial examples among the inputs in each layer, the learning rate for updating the input features in each layer and the dropout rate, respectively, influence the training and testing accuracies of AD-TSK. Owing to space limitations, Tables 6, 7 and 8 present the trends of change exhibited by the training and testing accuracies of AD-TSK for different values of β , λ and ρ on six benchmark datasets, which are randomly selected from the fourteen benchmark datasets. For clarity, when observing how one parameter influences the performance of AD-TSK, the other two parameters are fixed. Specifically, the results obtained under different values of β with $\lambda = 0.001$ and $\rho = 0.8$ are shown in Table 6, the results obtained under different values of λ with $\beta = 0.001$ and $\rho = 0.8$ are shown in Table 7, and the results obtained under different values of ρ with $\beta = 0.001$ and $\lambda = 0.001$ are shown in Table 8.

According to Table 6, when more realistic labels of the inputs for the ATSK in each layer are attacked in the training phase of AD-TSK, i.e., the percentage of proactive adversarial examples among the inputs of ATSK in each layer of AD-TSK is increased, the classification performance

improvement achieved by AD-TSK is not significant, and in particular, the testing accuracy decreases slightly. Extensive experimental results demonstrate that setting the percentage β of proactive adversarial examples among the inputs for each ATSK to 0.001 can guarantee that AD-TSK performs well in most cases.

In addition, Table 7 shows that both the training and testing accuracies of AD-TSK generally decrease with increasing learning rate λ . According to Steps 7 and 16 in Algorithm 1, the greater the values of λ are, the more gradient information possessed by the output errors induced by each ATSK with respect to the inputs is introduced to update the inputs of each ATSK. This gradually destroys the internal data structure of the original inputs along with the stacked structure of AD-TSK. Thus, small values of λ , such as 0.001, are more appropriate for constructing the stacked structure on the basis of the stacked generalization principle [19].

Moreover, according to Table 8, when $\rho = 0$, i.e., when all gradient information possessed by the output errors induced by the ATSK in each layer of AD-TSK with respect to the inputs is integrated into the input features to update the inputs of each ATSK in a layer-by-layer manner, AD-TSK encounters overfitting problem, especially on PHO and TIT. This issue is alleviated with increasing ρ . Therefore, the use of dropout is recommended to randomly select partial gradient information from the output errors induced by ATSK with respect to the inputs to update the inputs of each ATSK along with the stacked structure to guarantee the generalization capability of AD-TSK.

Table 6 Influences of different values of β on the classification performance of AD-TSK

Datasets	Indices	$\beta = 0.001$	$\beta = 0.01$	$\beta = 0.1$
PHO	Training	87.82 ± 1.47	87.50 ± 1.77	86.86 ± 1.01
	Testing	85.99 ± 0.69	84.69 ± 0.37	84.00 ± 0.36
TIT	Training	82.30 ± 1.71	83.03 ± 1.55	83.68 ± 1.59
	Testing	82.54 ± 0.28	81.56 ± 0.28	79.95 ± 0.19
PAG	Training	94.89 ± 0.68	94.54 ± 0.54	94.98 ± 0.49
	Testing	94.34 ± 0.33	94.01 ± 0.17	94.21 ± 0.35
TWO	Training	97.88 ± 0.11	97.92 ± 0.08	97.93 ± 0.14
	Testing	98.20 ± 0.07	98.03 ± 0.04	98.07 ± 0.09
NUR	Training	53.78 ± 1.91	53.83 ± 1.78	52.22 ± 1.30
	Testing	52.58 ± 1.80	52.39 ± 1.32	51.01 ± 1.05
MAG	Training	89.19 ± 1.01	90.35 ± 1.10	90.05 ± 1.13
	Testing	90.32 ± 0.25	88.98 ± 0.30	87.56 ± 0.39

Table 7 Influences of different values of λ on the classification performance of AD-TSK

Datasets	Indices	$\lambda = 0.001$	$\lambda = 0.01$	$\lambda = 0.1$
PHO	Training	87.82 ± 1.47	87.45 ± 1.47	87.12 ± 1.36
	Testing	85.99 ± 0.69	84.69 ± 0.37	84.00 ± 0.36
TIT	Training	82.30 ± 1.71	84.18 ± 2.54	82.44 ± 2.79
	Testing	82.54 ± 0.28	81.02 ± 0.22	78.87 ± 0.32
PAG	Training	94.89 ± 0.68	95.50 ± 0.63	93.07 ± 1.10
	Testing	94.34 ± 0.33	93.72 ± 0.42	92.38 ± 0.60
TWO	Training	97.88 ± 0.11	97.95 ± 0.08	97.91 ± 0.22
	Testing	98.20 ± 0.07	97.99 ± 0.06	97.95 ± 0.08
NUR	Training	53.78 ± 1.91	57.19 ± 1.95	45.34 ± 3.85
	Testing	52.58 ± 1.80	53.43 ± 1.30	44.85 ± 3.04
MAG	Training	89.19 ± 1.01	91.03 ± 1.08	89.61 ± 1.36
	Testing	90.32 ± 0.25	88.42 ± 0.43	87.78 ± 0.35

Table 8 Influences of different values of ρ on the classification performance of AD-TSK

Datasets	Indices	$\rho = 0$	$\rho = 0.2$	$\rho = 0.4$	$\rho = 0.6$	$\rho = 0.8$
PHO	Training	99.38 ± 0.55	96.94 ± 0.89	94.80 ± 1.18	90.50 ± 1.39	87.82 ± 1.47
	Testing	88.38 ± 0.19	87.32 ± 0.22	87.42 ± 0.28	87.21 ± 0.62	85.99 ± 0.69
TIT	Training	96.66 ± 2.52	95.92 ± 3.29	93.57 ± 2.21	87.36 ± 2.35	82.30 ± 1.71
	Testing	$81.86 \pm -$	83.17 ± 0.18	80.43 ± 0.30	81.97 ± 0.36	82.54 ± 0.28
PAG	Training	99.63 ± 0.17	98.06 ± 0.47	98.56 ± 0.57	96.96 ± 0.66	94.89 ± 0.68
	Testing	97.66 ± 0.05	96.57 ± 0.10	96.13 ± 0.10	95.54 ± 0.28	94.34 ± 0.33
TWO	Training	97.90 ± 0.15	97.89 ± 0.09	98.10 ± 0.10	97.66 ± 0.07	97.88 ± 0.11
	Testing	98.22 ± 0.12	98.05 ± 0.08	97.59 ± 0.09	98.04 ± 0.06	98.20 ± 0.07
NUR	Training	99.89 ± 0.16	92.07 ± 1.97	79.86 ± 4.22	68.82 ± 3.10	53.78 ± 1.91
	Testing	92.03 ± 0.04	84.81 ± 1.74	74.73 ± 3.51	64.97 ± 2.44	52.58 ± 1.80
MAG	Training	98.57 ± 1.98	95.92 ± 3.16	94.71 ± 1.03	91.46 ± 2.19	89.19 ± 1.01
	Testing	91.14 ± 0.19	89.64 ± 0.34	90.48 ± 0.21	89.16 ± 0.30	90.32 ± 0.25

Table 9 Average training times (seconds) required by AD-TSK and F-AD-TSK on all benchmark datasets

Datasets	AD-TSK	F-AD-TSK
LIV	0.74	0.17
PIM	8.02	3.17
RED	10.40	3.29
TIT	2.45	0.33
CHE	56.74	27.97
PHO	11.72	1.59
PAG	24.65	4.02
TWO	68.19	16.46
NUR	50.56	5.54
MAG	14.58	2.67
CCC	323.95	211.11
ADU	331.26	136.53
SHU	230.34	62.15
INT	333.47	142.45

Running time and interpretability analysis

To verify whether the inverse-free fast deep learning algorithm can quickly train AD-TSK, we list the average training times required by AD-TSK and F-AD-TSK on all benchmark datasets in Table 9. For a fair comparison, the parameters involved in F-AD-TSK are set to be the same as those in AD-TSK, i.e., $\beta = 0.001$, $\lambda = 0.001$, $\rho = 0$, and the number of layers is 5. In addition, the training times listed in Table 9 are the sums of the average running times utilized when training the AD-TSK and F-AD-TSK models using 2 to 20 fuzzy rules.

Table 9 shows that F-AD-TSK consumes less training time than AD-TSK on all benchmark datasets, which is consistent with the computational complexity analysis of the inverse-free fast deep learning algorithm of AD-TSK described in “Inverse-free fast deep learning algorithm of AD-TSK” section. In particular, F-AD-TSK clearly has shorter training time than AD-TSK with increasing sample size, such as the PAG, TWO, NUR, MAG, CCC, ADU, SHU and INT datasets. These observations prove the feasibility of introducing the Schur complement and Sherman–Morrison formula [20, 23, 24] to develop the inverse-free fast deep learning algorithm of AD-TSK.

To intuitively demonstrate the interpretability of AD-TSK, Table 10 presents the fuzzy rules generated by ATSK in each layer on LIV. In Table 10, m denotes the sequence number of each layer, and r denotes the sequence number of each fuzzy rule in each layer. For example, $m = 2$ together with $r = 1$ denotes the first fuzzy rule generated for ATSK in the second layer of AD-TSK. In addition, the features in each sample included in LIV are denoted as F_1, F_2, F_3, F_4, F_5 and F_6 , and

each feature corresponds to an interpretable linguistic label among $\{very\ low, low, medium, high, very\ high\}$, which are simplified as $\{VL, L, M, H, VH\}$, respectively.

According to Table 10, all fuzzy rules in AD-TSK can be translated into fuzzy rules with comprehensible IF-parts and linear THEN-parts, which can guarantee the interpretability of AD-TSK. The first fuzzy rule produced for ATSK in the second layer of AD-TSK is taken as an example. This fuzzy rule can be translated into an interpretable fuzzy rule, which is described as

IF F_1 is $VH \wedge F_2$ is $VL \wedge F_3$ is $H \wedge F_4$ is $M \wedge F_5$ is $VH \wedge F_6$ is M ,

THEN $f(\mathbf{x}) = -91.87 + 4.50 * F_1 + 5.14 * F_2 - 44.56 * F_3 + 23.95 * F_4 + 20.97 * F_5 + 29.18 * F_6$.

Limitations of the proposed AD-TSK

Since the stacked structure of the proposed AD-TSK is built by taking the TSK fuzzy system [6, 46] as the basic unit in each layer based on conducting proactive adversarial example learning, some limitations of the TSK fuzzy system are inherited by AD-TSK. For example, the rule explosion problem, where the number of fuzzy rules in a TSK fuzzy system grows exponentially with the number of input features and fuzzy sets [6], is potentially magnified by AD-TSK because of the stacked structure, and the linear consequent problem, where the TSK model uses linear functions in the consequent parts of the fuzzy rules, may result in nonlinear relationships not being captured adequately in complex systems [47]. In the near future, we will further investigate the performance of AD-TSK to address these problems and improve the feasibility of AD-TSK in real-world applications.

Conclusion and future directions

In this paper, a proactive adversarial example learning-based deep TSK fuzzy classifier, AD-TSK, is proposed to achieve encouraging classification performance while maintaining high interpretability. AD-TSK is characterized by its novel stacked structure, which is built through proactive adversarial example learning and the dropout-based inputs update in a layer-by-layer manner. The novel stacked structure enables us to enhance the robustness of AD-TSK to noise data, which improves the generalization capability of AD-TSK. In addition, all the input features correspond to their interpretable linguistic labels, and the antecedent part of each fuzzy rule is generated by randomly combining Gaussian membership functions [36], which effectively guarantees the interpretability of AD-TSK. In particular, owing to the introduction of the Schur complement and Sherman–Morrison formula [20, 23, 24], an inverse-free fast deep learning algorithm is devel-

Table 10 Illustration of the fuzzy rules produced in each layer of AD-TSK on LIV

m	r	IF-parts						THEN-parts
		F_1	F_2	F_3	F_4	F_5	F_6	
1	1	<i>VH</i>	<i>L</i>	<i>VL</i>	<i>M</i>	<i>M</i>	<i>VH</i>	[−63.26, 8.19, 8.81, 57.53, −29.45, 21.05, 60.51]
	2	<i>H</i>	<i>VH</i>	<i>VH</i>	<i>H</i>	<i>VH</i>	<i>H</i>	[19.64, −98.18, −25.68, −5.35, −121.98, −84.71, 194.36]
	3	<i>VL</i>	<i>L</i>	<i>VL</i>	<i>M</i>	<i>VL</i>	<i>VL</i>	[12.86, 98.86, −1.66, −12.06, 43.19, 34.13, −107.37]
2	1	<i>VH</i>	<i>VL</i>	<i>H</i>	<i>M</i>	<i>VH</i>	<i>M</i>	[−91.87, 4.50, 5.14, −44.56, 23.95, 20.97, 29.18]
	2	<i>VH</i>	<i>M</i>	<i>VH</i>	<i>L</i>	<i>VH</i>	<i>H</i>	[92.11, −1.98, −26.32, −53.06, −30.72, −34.84, 82.58]
	3	<i>VH</i>	<i>VH</i>	<i>L</i>	<i>L</i>	<i>M</i>	<i>H</i>	[77.19, −40.55, 16.50, −21.92, −57.70, −0.28, −31.90]
3	1	<i>M</i>	<i>L</i>	<i>VL</i>	<i>VH</i>	<i>VL</i>	<i>M</i>	[−121.23, 48.70, 43.49, −8.63, 22.78, −121.33, −9.73]
	2	<i>VH</i>	<i>L</i>	<i>VL</i>	<i>L</i>	<i>VH</i>	<i>H</i>	[−16.27, 7.82, −2.90, 23.20, −11.93, 13.75, 5.18]
	3	<i>M</i>	<i>M</i>	<i>VH</i>	<i>VL</i>	<i>L</i>	<i>VH</i>	[113.73, −22.83, 10.43, −44.27, 20.76, −76.27, 53.79]
4	1	<i>VL</i>	<i>H</i>	<i>L</i>	<i>VL</i>	<i>H</i>	<i>VL</i>	[2.99, 13.06, −21.50, 18.13, 16.96, −7.13, 23.64]
	2	<i>L</i>	<i>M</i>	<i>VL</i>	<i>M</i>	<i>M</i>	<i>L</i>	[43.29, 26.39, −38.75, −15.93, 2.63, −6.04, 4.20]
	3	<i>M</i>	<i>L</i>	<i>L</i>	<i>VH</i>	<i>H</i>	<i>VL</i>	[−70.07, 20.39, 24.44, 36.17, −3.87, −55.25, −5.40]
5	1	<i>VH</i>	<i>VH</i>	<i>H</i>	<i>L</i>	<i>M</i>	<i>H</i>	[−43.73, 13.70, 37.80, 118.17, −73.54, −28.16, 93.08]
	2	<i>M</i>	<i>H</i>	<i>H</i>	<i>VL</i>	<i>L</i>	<i>VL</i>	[−124.20, 28.59, 8.66, −44.63, 93.88, −5.60, 0.85]
	3	<i>VH</i>	<i>VH</i>	<i>VL</i>	<i>VL</i>	<i>VL</i>	<i>H</i>	[82.71, 13.26, −38.93, −83.56, 44.99, 10.46, −59.92]

oped to realize fast training for the AD-TSK model. The experimental results obtained on benchmark datasets and the statistical test results demonstrate the advantages of AD-TSK and the feasibility of its fast deep learning algorithm.

In the future, we will continue to explore three main aspects of the proposed AD-TSK model. First, although proactive adversarial example learning can improve the robustness of AD-TSK to noise data, how to adaptively set the percentage of proactive adversarial examples among the inputs in each layer according to the given data distribution to address the noise data precisely is worth studying. Second, since there is an equivalence between fuzzy rule-based systems and feed-forward neural networks [20, 36], we attempt to design a fuzzy convolutional layer based on the TSK fuzzy system to combine with or even substitute for the ATSK in each layer and enable AD-TSK to perform large-scale image analysis. Third, as noted in “Limitations of the proposed AD-TSK” section, we focus on addressing the limitations of AD-TSK inherited from the TSK fuzzy system [6] by introducing rule reduction technologies such as genetic algorithms and incorporating nonlinear consequent functions or hybrid models such as the neuro-fuzzy system [2] to increase the applicability of AD-TSK.

Author contributions Contributions Limin Mao: Software, Investigation, Resources, Data curation, Writing—original draft, Visualization. Hangmin Shi: Methodology, Validation, Writing—review and editing, Supervision. Yongxin Chou, Jinliang Cong, Mingli Lu, Zekang Bian: Validation, Writing—review and editing, Supervision. Suhang Gu: Conceptualization, Methodology, Validation, Formal analysis, Writing—review and editing, Supervision, Project administration, Funding acquisition.

Funding This work was supported in part by the National Natural Science Foundation of China under Grants 62106025, 62241102 and 62306126, in part by the Jiangsu Province 333 High-Level Talent Cultivation Project under Grant 2024-3-0484, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20210940, in part by Suzhou Municipal Science and Technology Plan Project under Grants SYG202129 and SYG202351, in part by Zhangjiagang Municipal Science and Technology Plan Project under Grant ZKYY2222, and in part by Qinglan Project of Jiangsu Province.

Data availability All the datasets involved in this paper can be downloaded from UCI and KEEL data repositories.

Code availability The source code of the proposed AD-TSK can be downloaded from <https://github.com/gusuhang10/AD-TSK>.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical approval The article has the approval of all the authors.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission

directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Zhao J, Wang H, Yao W et al (2024) An online surrogate-assisted neighborhood search algorithm based on deep neural network for thermal layout optimization. *Complex Intell Syst* 10(2):2459–2475. <https://doi.org/10.1007/s40747-023-01276-0>
- Deng Y, Ren Z, Kong Y et al (2017) A hierarchical fused fuzzy deep neural network for data classification. *IEEE Trans Fuzzy Syst* 25(4):1006–1012. <https://doi.org/10.1109/TFUZZ.2016.2574915>
- Zhang L, Zhang J, Gao W et al (2024) A deep learning outline aimed at prompt skin cancer detection utilizing gated recurrent unit networks and improved orca predation algorithm. *Biomed Signal Process Control* 90:105858. <https://doi.org/10.1016/j.bspc.2023.105858>
- Salimi Badr A, Ebadzadeh MM (2022) A novel self-organizing fuzzy neural network to learn and mimic habitual sequential tasks. *IEEE Trans Cybern* 52(1):323–332. <https://doi.org/10.1109/TCYB.2020.2984646>
- Roh SB, Oh SK, Pedrycz W et al (2022) Design of iterative fuzzy radial basis function neural networks based on iterative weighted fuzzy c-means clustering and weighted LSE estimation. *IEEE Trans Fuzzy Syst* 30(10):4273–4285. <https://doi.org/10.1109/TFUZZ.2022.3146985>
- Zhang Y, Wang G, Zhou T et al (2024) Takagi–Sugeno–Kang fuzzy system fusion: a survey at hierarchical, wide and stacked levels. *Inf Fusion* 101:101977. <https://doi.org/10.1016/j.inffus.2023.101977>
- Mosca A, Magoulas GD (2018) Hardening against adversarial examples with the smooth gradient method. *Soft Comput* 22(10):3203–3213. <https://doi.org/10.1007/s00500-017-2998-4>
- Dai J, Luo L, Xiao L et al (2024) Modified noise-immune fuzzy neural network for solving the quadratic programming with equality constraint problem. *IEEE Trans Neural Netw Learn Syst* 35(11):15825–15833. <https://doi.org/10.1109/TNNLS.2023.3290030>
- Liu H, Ghadimi N (2024) Hybrid convolutional neural network and flexible dwarf mongoose optimization algorithm for strong kidney stone diagnosis. *Biomed Signal Process Control* 91:106024. <https://doi.org/10.1016/j.bspc.2024.106024>
- Philip CCL, Zhang C, Chen L et al (2015) Fuzzy restricted Boltzmann machine for the enhancement of deep learning. *IEEE Trans Fuzzy Syst* 23(6):2163–2173. <https://doi.org/10.1109/TFUZZ.2015.2406889>
- Li Y, Gault R, McGinnity TM (2022) Probabilistic, recurrence, fuzzy neural network for processing noisy time-series data. *IEEE Trans Neural Netw Learn Syst* 33(9):4851–4860. <https://doi.org/10.1109/TNNLS.2021.3061432>
- Qin B, Nojima Y, Ishibuchi H et al (2021) Realizing deep high-order tsf fuzzy classifier by ensembling interpretable zero-order tsf fuzzy subclassifiers. *IEEE Trans Fuzzy Syst* 29(11):3441–3455. <https://doi.org/10.1109/TFUZZ.2020.3022574>
- Zhang Y, Ishibuchi H, Wang S (2018) Deep Takagi–Sugeno–Kang fuzzy classifier with shared linguistic fuzzy rules. *IEEE Trans Fuzzy Syst* 26(3):1535–1549. <https://doi.org/10.1109/TFUZZ.2017.2729507>
- Gu S, Chung FL, Wang S (2020) A novel deep fuzzy classifier by stacking adversarial interpretable tsf fuzzy sub-classifiers with smooth gradient information. *IEEE Trans Fuzzy Syst* 28(7):1369–1382. <https://doi.org/10.1109/TFUZZ.2019.2919481>
- Zhou T, Ishibuchi H, Wang S (2018) Stacked blockwise combination of interpretable tsf fuzzy classifiers by negative correlation learning. *IEEE Trans Fuzzy Syst* 26(6):3327–3341. <https://doi.org/10.1109/TFUZZ.2018.2824763>
- Zhou T, Chung FL, Wang S (2017) Deep tsf fuzzy classifier with stacked generalization and triply concise interpretability guarantee for large data. *IEEE Trans Fuzzy Syst* 25(5):1207–1221. <https://doi.org/10.1109/TFUZZ.2016.2604003>
- Gu S, Vong CM, Wong PK et al (2022) Fast training of adversarial deep fuzzy classifier by downsizing fuzzy rules with gradient guided learning. *IEEE Trans Fuzzy Syst* 30(6):1967–1980. <https://doi.org/10.1109/TFUZZ.2021.3072498>
- Wang Y, Liu H, Jia W et al (2022) Deep fuzzy rule-based classification system with improved Wang–Mendel method. *IEEE Trans Fuzzy Syst* 30(8):2957–2970. <https://doi.org/10.1109/TFUZZ.2021.3098339>
- Sigletos G, Paliouras G, Spyropoulos CD et al (2005) Combining information extraction systems using voting and stacked generalization. *J Mach Learn Res* 6:1751–1782
- Li S, You Z, Guo H et al (2016) Inverse-free extreme learning machine with optimal information updating. *IEEE Trans Cybern* 46(5):1229–1241. <https://doi.org/10.1109/TCYB.2015.2434841>
- Zhou S, Liu C, Ye D et al (2022) Adversarial attacks and defenses in deep learning: from a perspective of cybersecurity. *ACM Comput Surv* 55(8):163. <https://doi.org/10.1145/3547330>
- Saha A, Subramanya A, Pirsiavash H (2020) Hidden trigger backdoor attacks. In: *Proceedings of the AAAI conference on artificial intelligence*, New York, USA, pp 11957–11965. <https://doi.org/10.1609/aaai.v34i07.6871>
- Bru R, Cerdán J, Marín J et al (2003) Preconditioning sparse nonsymmetric linear systems with the Sherman–Morrison formula. *SIAM J Sci Comput* 25(2):701–715. <https://doi.org/10.1137/S1064827502407524>
- Saad Y, Sosonkina M (1999) Distributed Schur complement techniques for general sparse linear systems. *SIAM J Sci Comput* 21(4):1337–1356. <https://doi.org/10.1137/S1064827597328996>
- Leski JM (2005) TSK-fuzzy modeling based on /spl epsiv/-insensitive learning. *IEEE Trans Fuzzy Syst* 13(2):181–193. <https://doi.org/10.1109/TFUZZ.2004.840094>
- Deng Z, Cao Y, Lou Q et al (2022) Monotonic relation-constrained Takagi–Sugeno–Kang fuzzy system. *Inf Sci* 582:243–257. <https://doi.org/10.1016/j.ins.2021.09.015>
- Diamantis DE, Iakovidis DK (2021) Fuzzy pooling. *IEEE Trans Fuzzy Syst* 29(11):3481–3488. <https://doi.org/10.1109/TFUZZ.2020.3024023>
- Mamaghani AS, Pedrycz W (2020) Structural optimization of fuzzy rule-based models: towards efficient complexity management. *Expert Syst Appl* 152:113362. <https://doi.org/10.1016/j.eswa.2020.113362>
- Gu S, Chou Y, Zhou J et al (2024) Takagi–Sugeno–Kang fuzzy clustering by direct fuzzy inference on fuzzy rules. *IEEE Trans Emerg Top Comput Intell* 8(2):1264–1279. <https://doi.org/10.1109/TETCI.2023.3336537>
- Huang G, Zhu Q, Siew CK (2006) Extreme learning machine: theory and applications. *Neurocomputing* 70(1–3):489–501. <https://doi.org/10.1016/j.neucom.2005.12.126>
- Júnior AHdS, Corona F, Barreto GA et al (2015) Minimal learning machine: a novel supervised distance-based approach for regression and classification. *Neurocomputing* 164:34–44. <https://doi.org/10.1016/j.neucom.2014.11.073>
- Zhao Y, Zhu H, Liang R et al (2019) Seeing isn't believing: towards more robust adversarial attack against real world object detectors. In: *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, London, UK, pp 1989–2004. <https://doi.org/10.1145/3319535.3354259>
- Grosse K, Papernot N, Manoharan P et al (2016) Adversarial perturbations against deep neural networks for malware classification. *CoRR abs/1606.04435*. [arXiv:1606.04435](https://arxiv.org/abs/1606.04435)

34. Shen X, Tian X, Liu T et al (2018) Continuous dropout. *IEEE Trans Neural Netw Learn Syst* 29(9):3926–3937. <https://doi.org/10.1109/TNNLS.2017.2750679>
35. Srivastava N, Hinton G, Krizhevsky A et al (2014) Dropout: A simple way to prevent neural networks from overfitting. *J Mach Learn Res* 15(56):1929–1958
36. Wong SY, Yap KS, Yap HJ et al (2015) On equivalence of FIS and ELM for interpretable rule-based knowledge representation. *IEEE Trans Neural Netw Learn Syst* 26(7):1417–1430. <https://doi.org/10.1109/TNNLS.2014.2341655>
37. Zhu H, Wang X, Wang R (2022) Fuzzy monotonic k-nearest neighbor versus monotonic fuzzy k-nearest neighbor. *IEEE Trans Fuzzy Syst* 30(9):3501–3513. <https://doi.org/10.1109/TFUZZ.2021.3117450>
38. Li CN, Li Y, Shao YH (2024) Large-scale structured output classification via multiple structured support vector machine by splitting. *IEEE Trans Emerg Top Comput Intell* 8(2):2112–2124. <https://doi.org/10.1109/TETCI.2024.3360339>
39. Pang L, Tay KM, Lim CP (2016) Monotone fuzzy rule relabeling for the zero-order TSK fuzzy inference system. *IEEE Trans Fuzzy Syst* 24(6):1455–1463. <https://doi.org/10.1109/TFUZZ.2016.2540059>
40. Deng Z, Choi KS, Chung FL et al (2011) Scalable TSK fuzzy modeling for very large datasets using minimal-enclosing-ball approximation. *IEEE Trans Fuzzy Syst* 19(2):210–226. <https://doi.org/10.1109/TFUZZ.2010.2091961>
41. Alata M, Su C, Demirli K (2001) Adaptive control of a class of nonlinear systems with a first-order parameterized Sugeno fuzzy approximator. *IEEE Trans Syst Man Cybern Part C* 31(3):410–419. <https://doi.org/10.1109/5326.971670>
42. Deng Z, Cao L, Jiang Y et al (2015) Minimax probability TSK fuzzy system classifier: a more transparent and highly interpretable classification model. *IEEE Trans Fuzzy Syst* 23(4):813–826. <https://doi.org/10.1109/TFUZZ.2014.2328014>
43. Frank A, Asuncion A (2010) UCI machine learning repository. Figshare <http://archive.ics.uci.edu/ml>
44. AlcaláFdez J, Fernández A, Luengo J et al (2011) KEEL data-mining software tool: data set repository, integration of algorithms and experimental analysis framework. *J Multiple Valued Log Soft Comput* 17:255–287
45. Demšar J (2006) Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res* 7:1–30
46. You W, Xie X, Wang H et al (2024) Relaxed model predictive control of T-S fuzzy systems via a new switching-type homogeneous polynomial technique. *IEEE Trans Fuzzy Syst* 32(8):4583–4594. <https://doi.org/10.1109/TFUZZ.2024.3405078>
47. Du Z, Xie X, Qu Z et al (2024) Dynamic event-triggered consensus control for interval type-2 fuzzy multi-agent systems. *IEEE Trans Circuits Syst I Regul Pap* 71(8):3857–3866. <https://doi.org/10.1109/TCSI.2024.3371492>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.